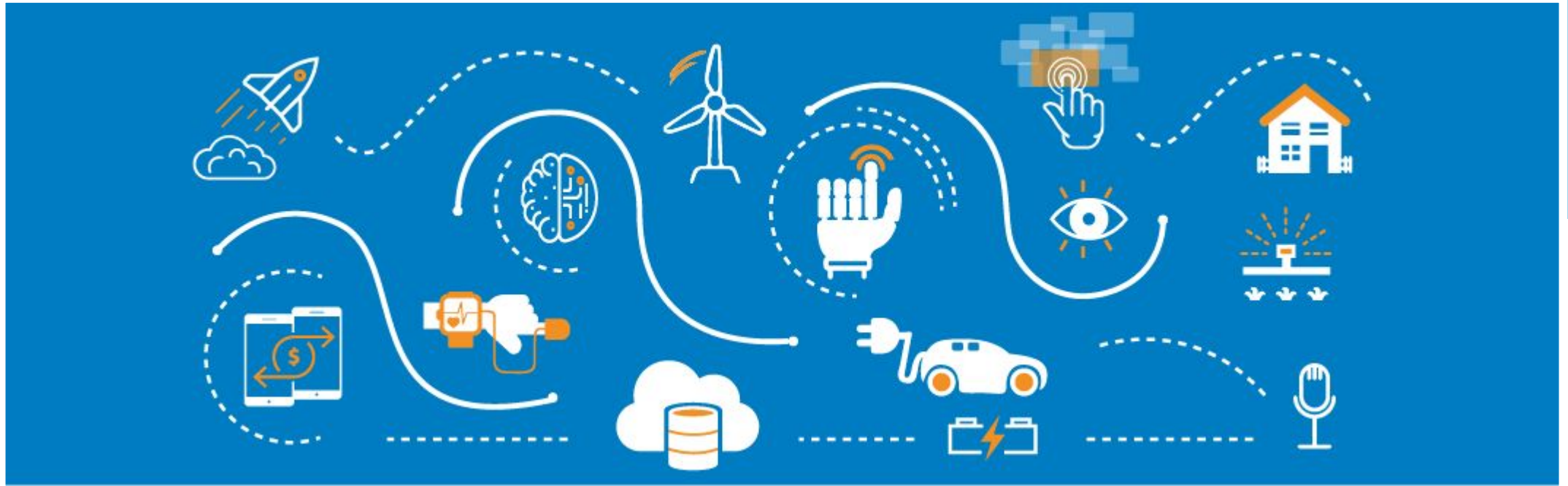




Natural Language Processing

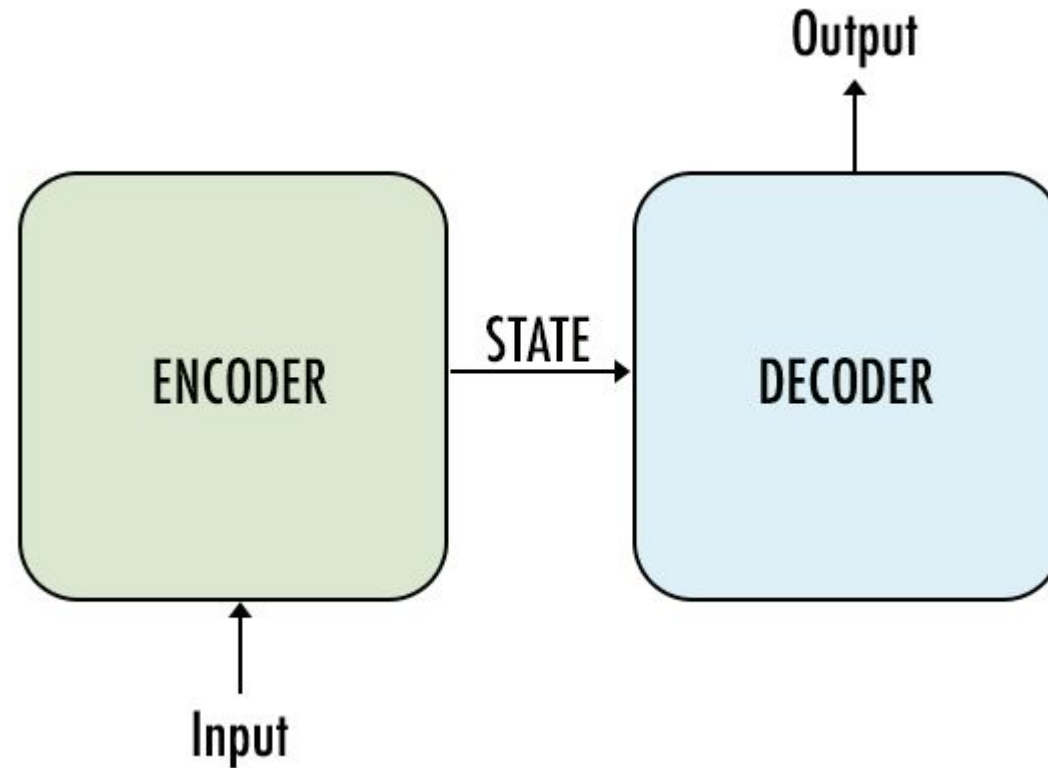
Session 2

Agenda

- 1** **Encoder - Decoder**
- 2** **Attention - Motivation**
- 3** **Transformer - Attention**
- 4** **BERT**
- 5** **Hands-On**

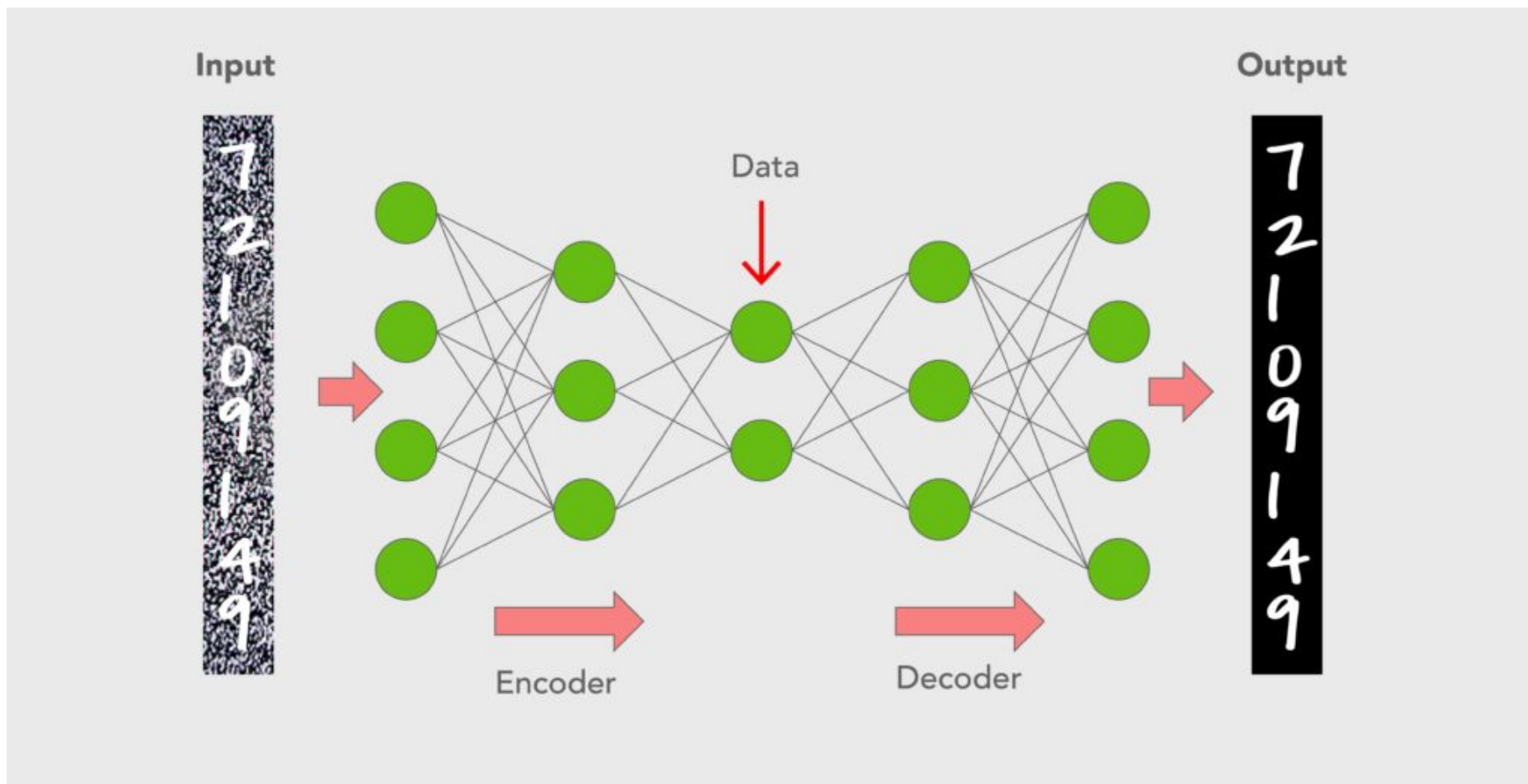
Encoder - Decoder

Encoder - Decoder Architecture



- **Encoder** - transforms input data into a different representation
- **Decoder** - converts representation into the desired output

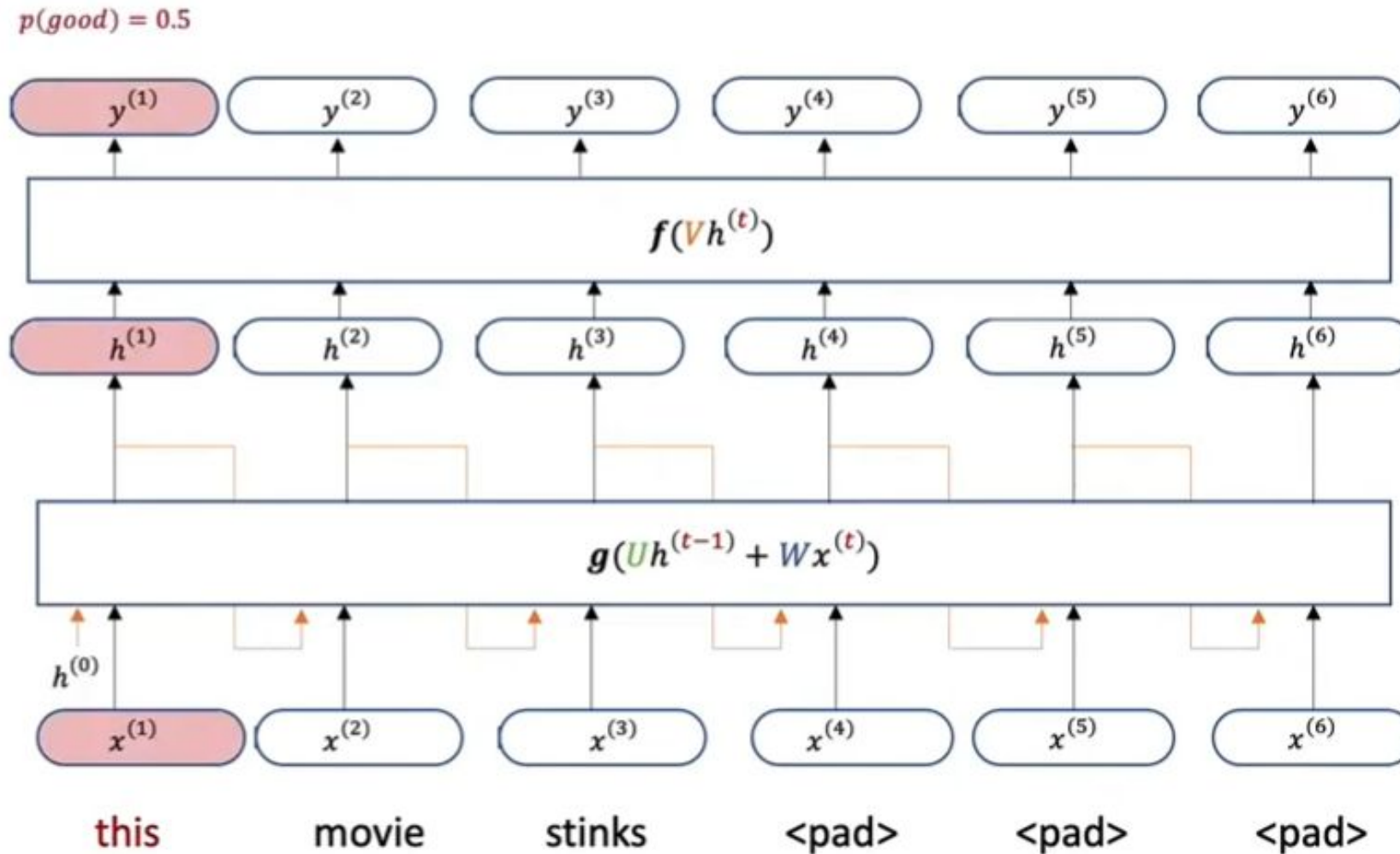
AutoEncoder



- Data compression
- Dimensionality reduction
- Anomaly detection
- Facial recognition
- Image reconstruction

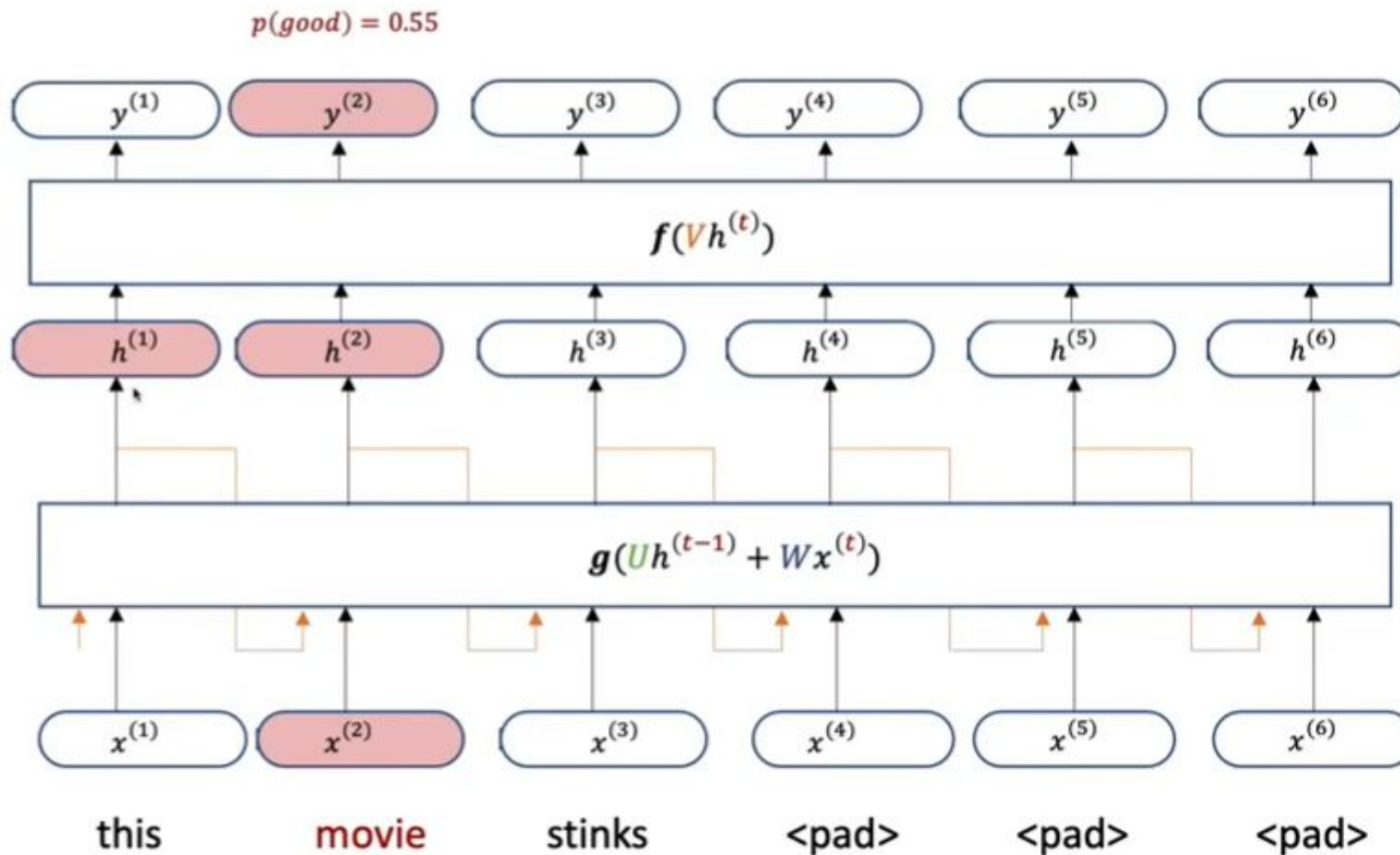
- An autoencoder is a type of artificial neural network used to learn efficient codings of unlabeled data
- Though all autoencoder models include both an encoder and a decoder, not all encoder-decoder models are autoencoders

Encoder Decoder Architecture in NLP



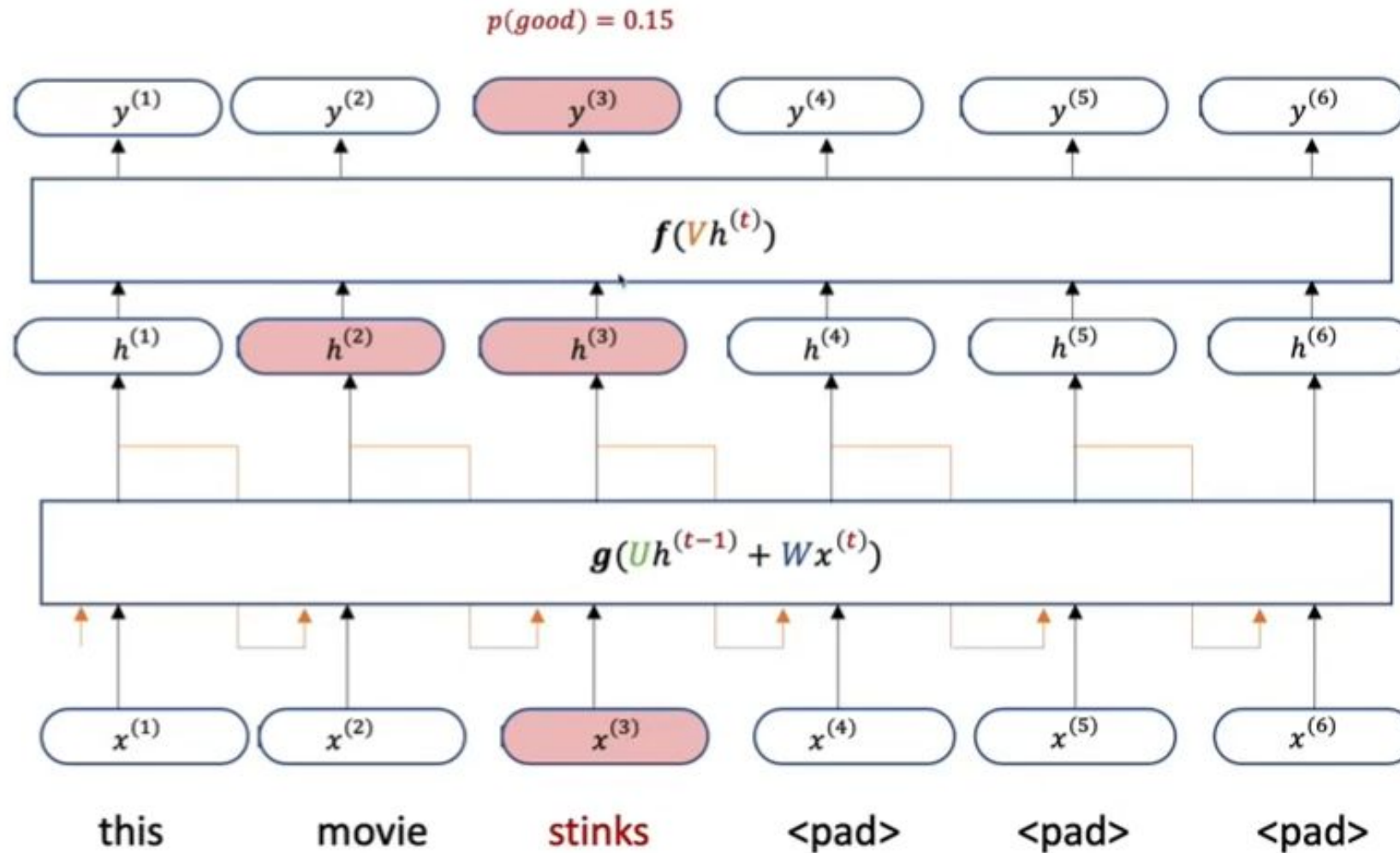
Example Movie Review Classification

Encoder Decoder Architecture in NLP



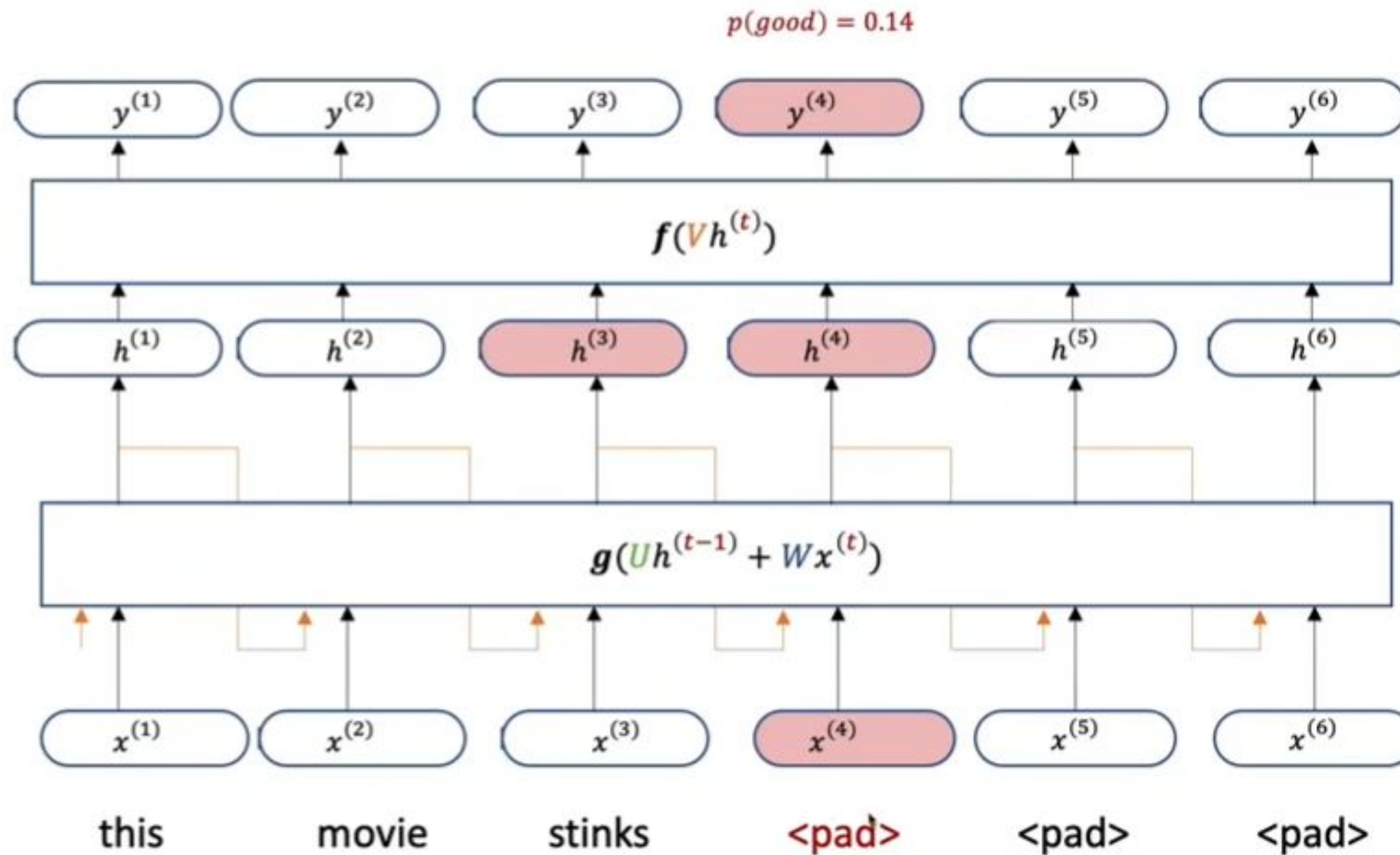
Example Movie Review Classification

Encoder Decoder Architecture in NLP



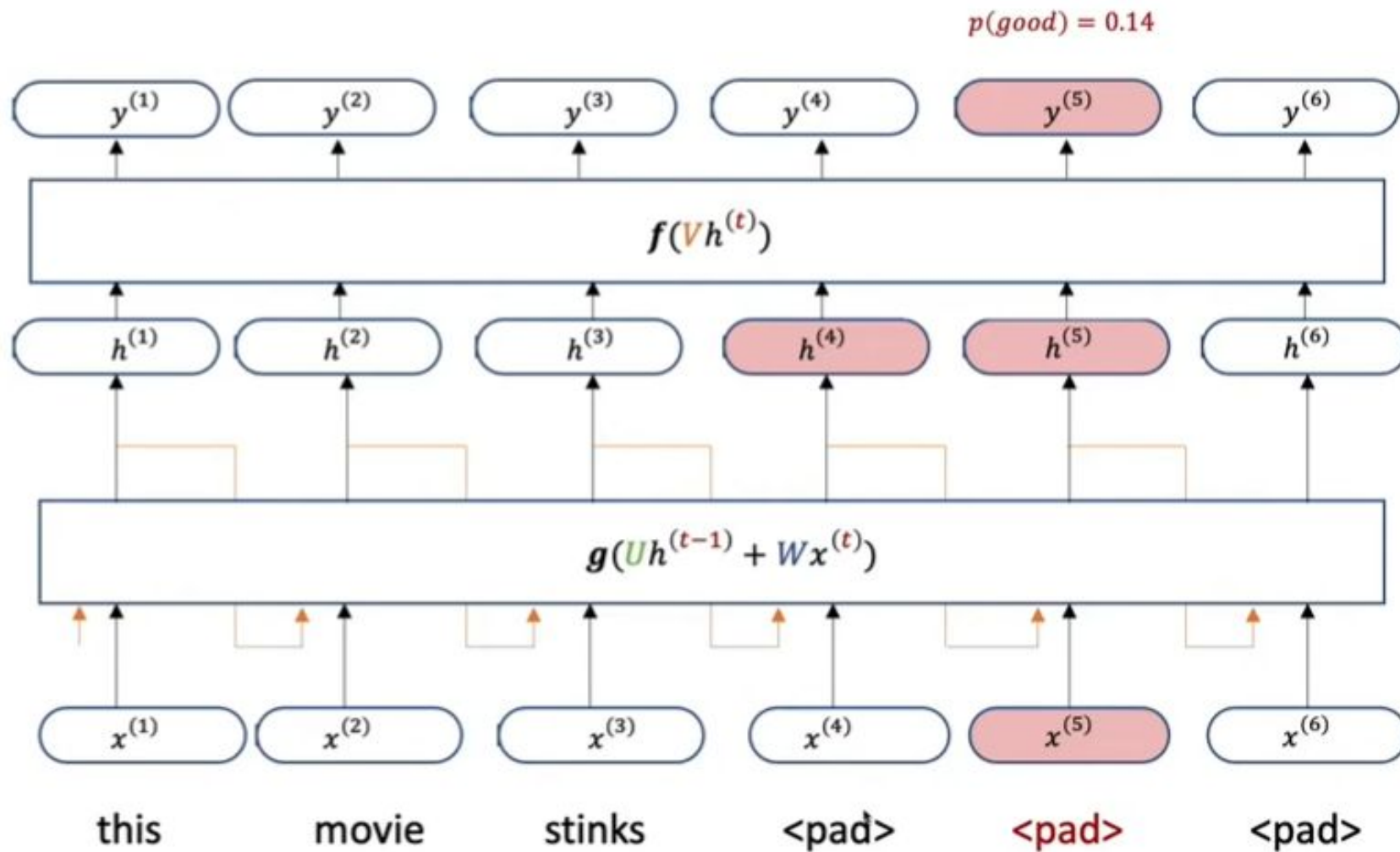
Example Movie Review Classification

Encoder Decoder Architecture in NLP



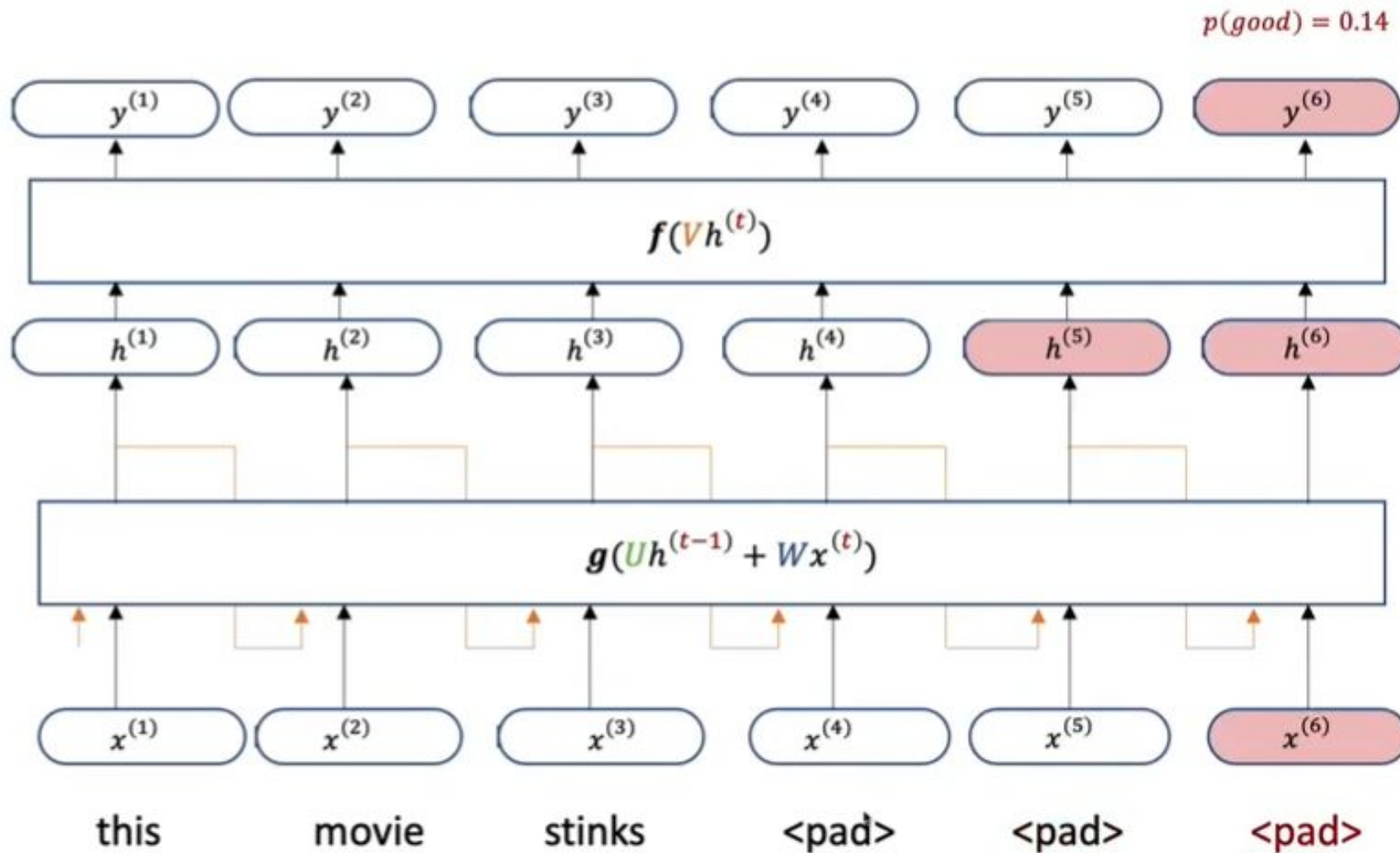
Example Movie Review Classification

Encoder Decoder Architecture in NLP



Example Movie Review Classification

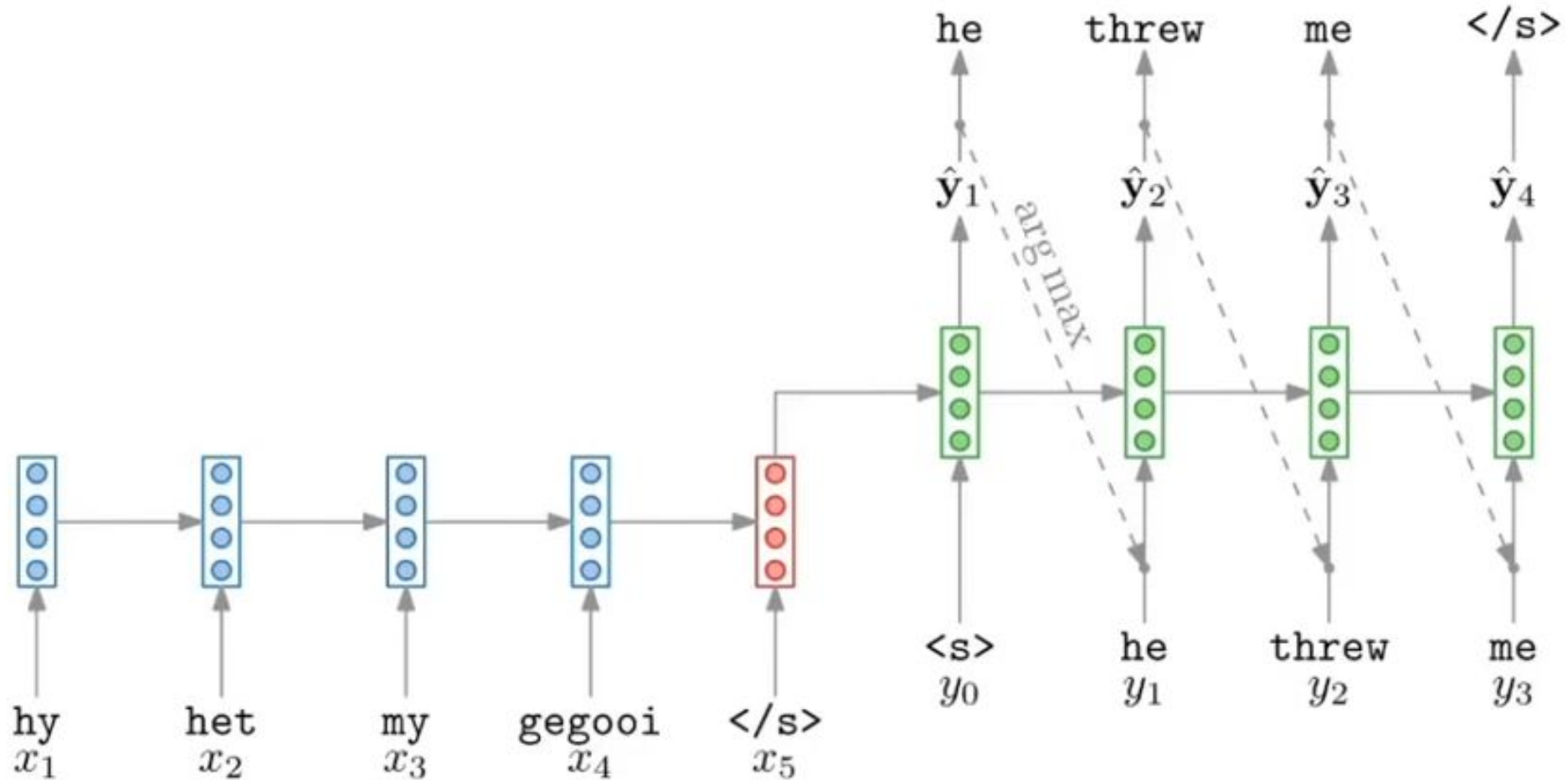
Encoder Decoder Architecture in NLP



Example Movie Review Classification

Encoder Decoder Architecture in NLP

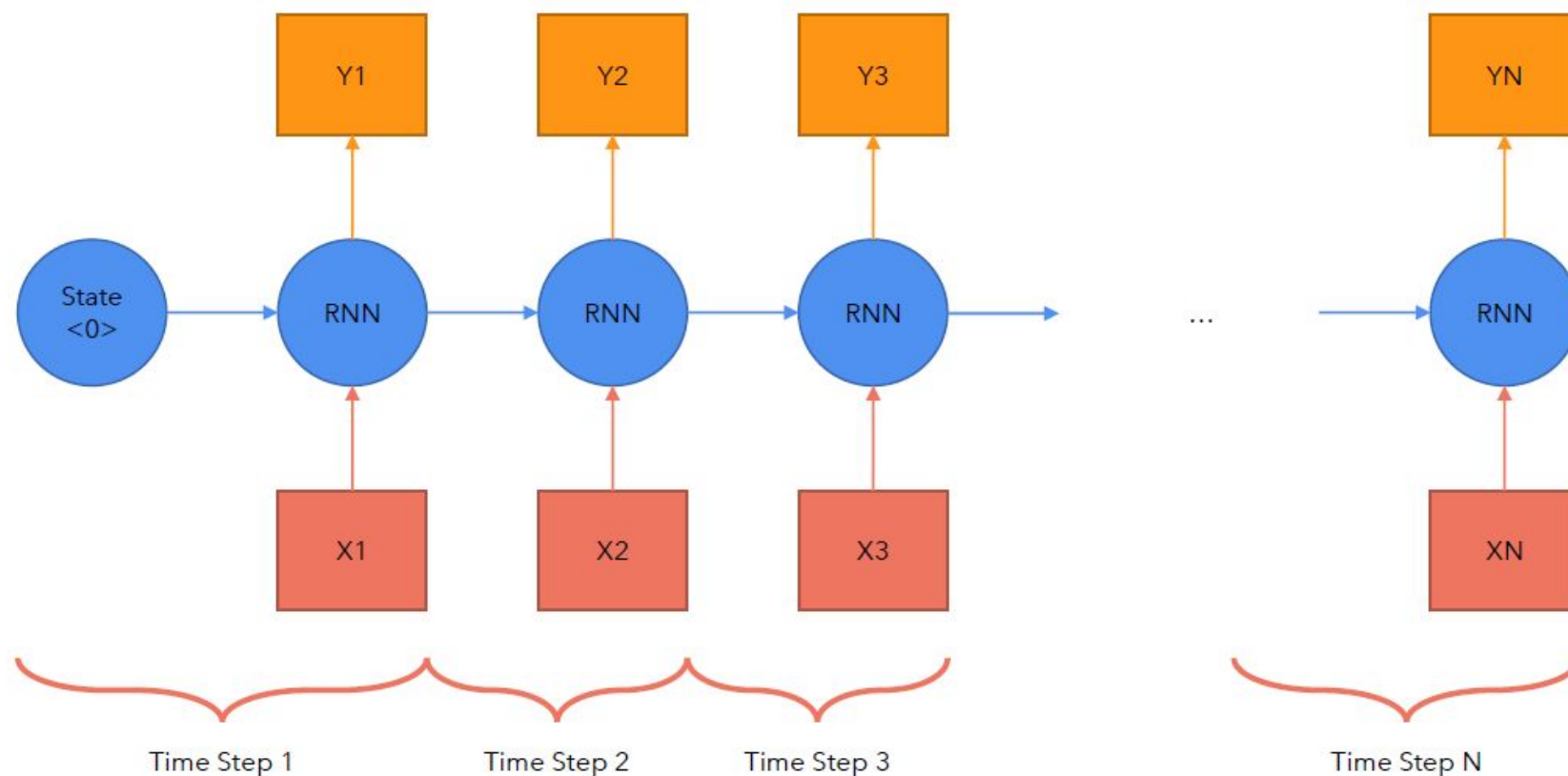
Netherlands → English



Example Encoder-Decoder for Machine Translation

Attention - Motivation

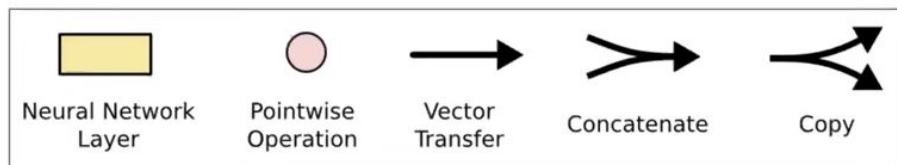
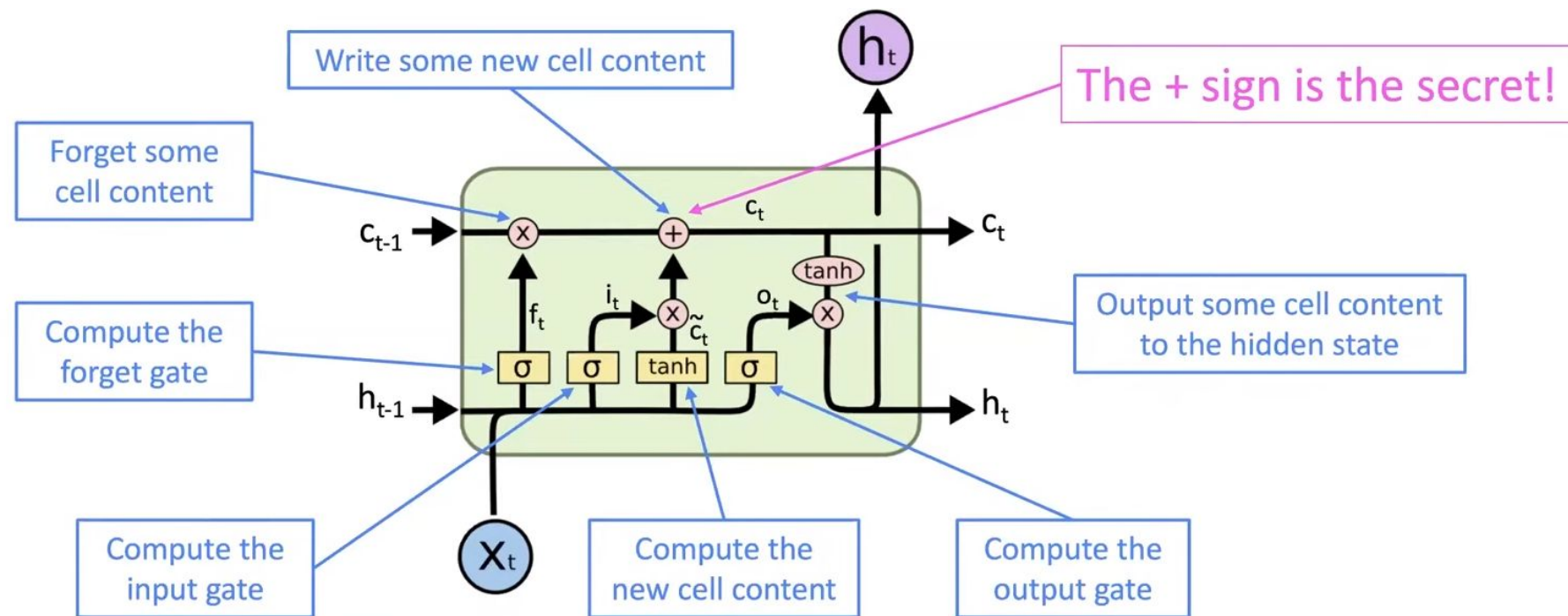
Recurrent Neural Networks (RNN)



Cons

- Slow computation for long sequence
- Vanishing or Exploding gradient
- Difficulty in accessing information from long time ago

Long Short Term Memory (LSTM)



Solution for vanishing/exploding

- Add direct connection

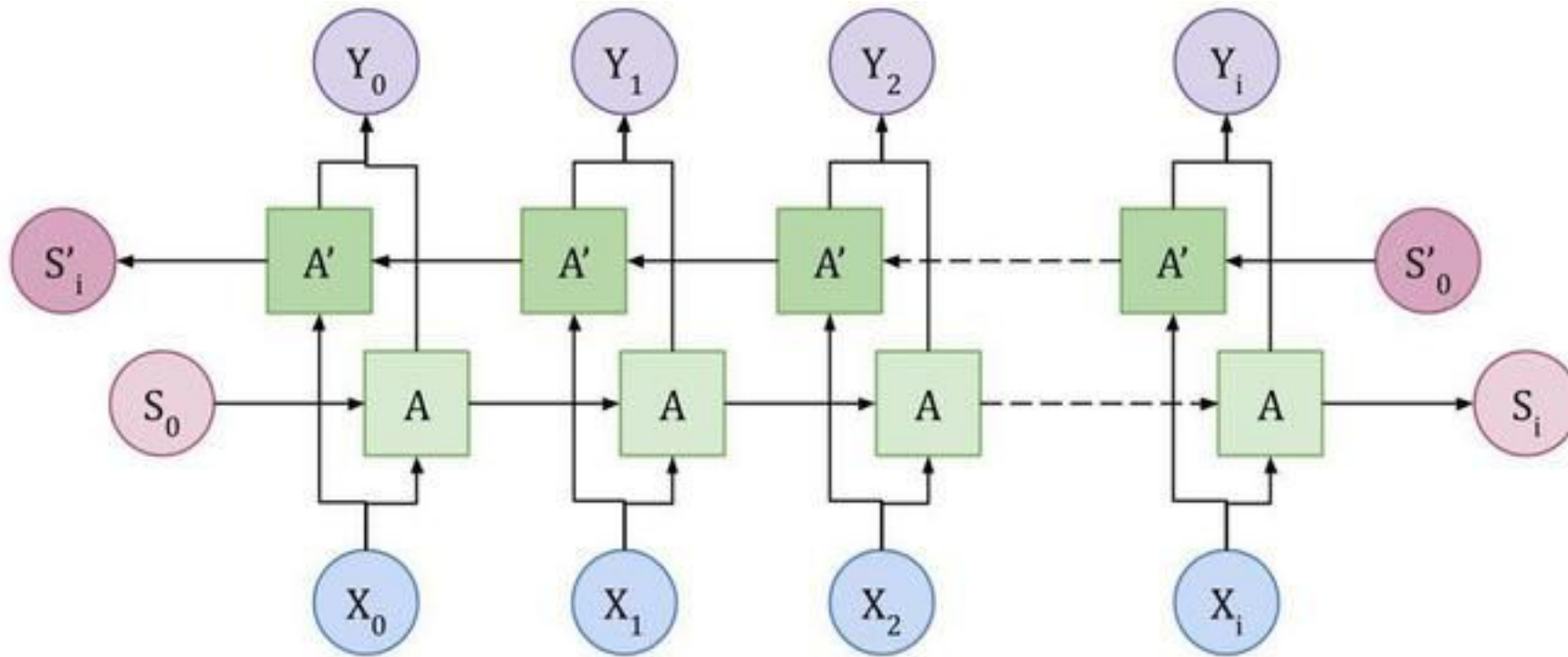
Pros

- Easier preserve information over many timesteps (forget gate 1, input gate 0, self information indefinitely)
- Remember 100 timesteps back than RNN (7)

Cons

- Doesn't guarantee no vanishing or exploding gradient
- Slow computation for long sequence
- Not actually learn the context, only from left to right

Bidirectional LSTM (BiLSTM)

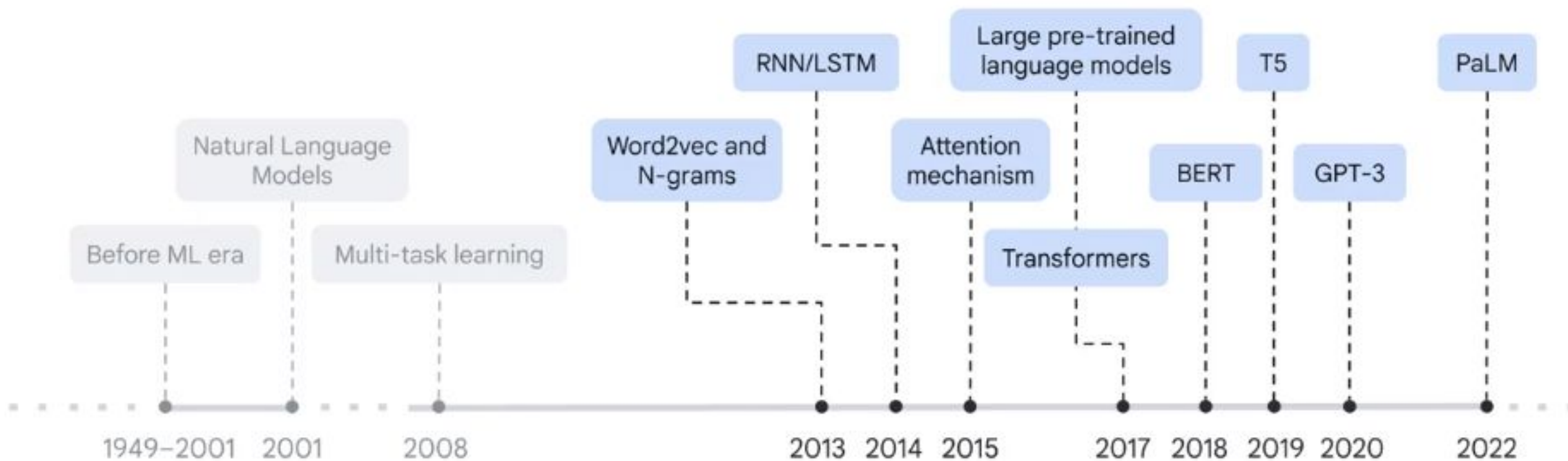


Cons

- Not actually learn the context even bidirectional LSTM

Transformer - Attention

Language Modeling History



Language Modeling History

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

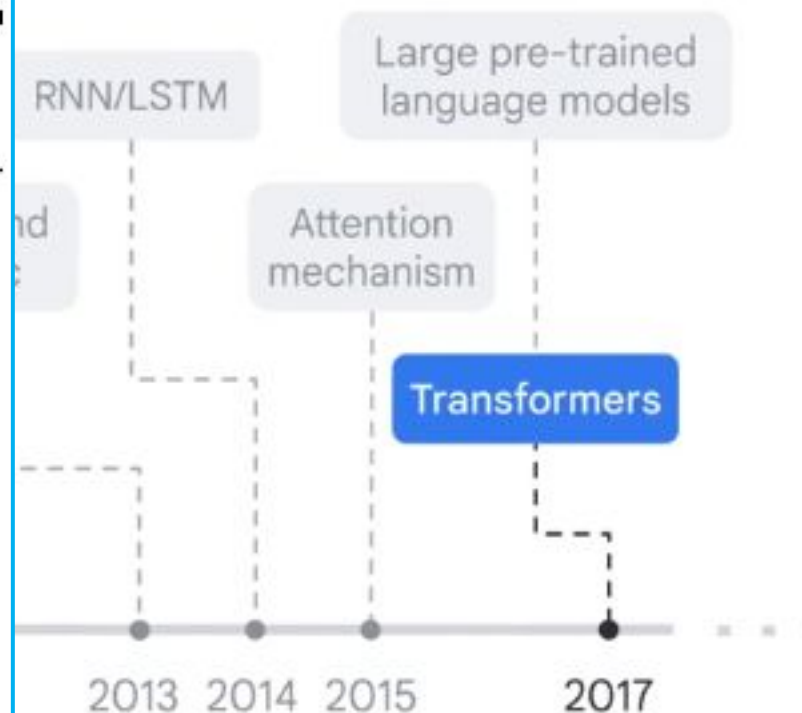
Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com



<https://arxiv.org/pdf/1706.03762.pdf>

Language Modeling History

LSTMs: real-world success

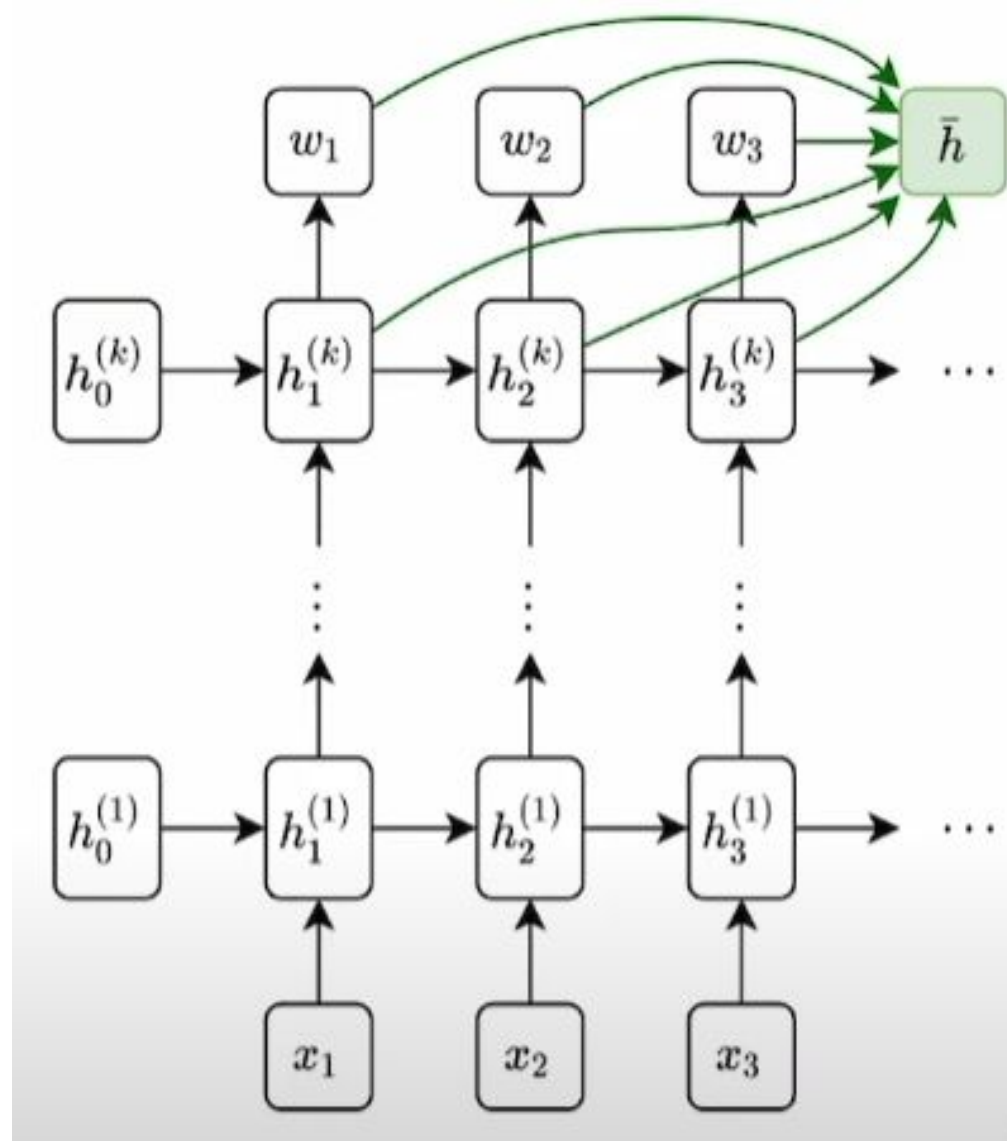
- In **2013–2015**, LSTMs started achieving state-of-the-art results
 - Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
 - LSTMs became the **dominant approach** for most NLP tasks
- **Now (2019–2024)**, **Transformers** have become dominant for all tasks
 - For example, in **WMT** (a Machine Translation conference + competition):
 - In WMT 2014, there were 0 neural machine translation systems (!)
 - In **WMT 2016**, the summary report contains “**RNN**” 44 times (and these systems won)
 - In WMT 2019: “**RNN**” 7 times, “**Transformer**” 105 times

Source: "Findings of the 2016 Conference on Machine Translation (WMT16)", Bojar et al. 2016, <http://www.statmt.org/wmt16/pdf/W16-2301.pdf>

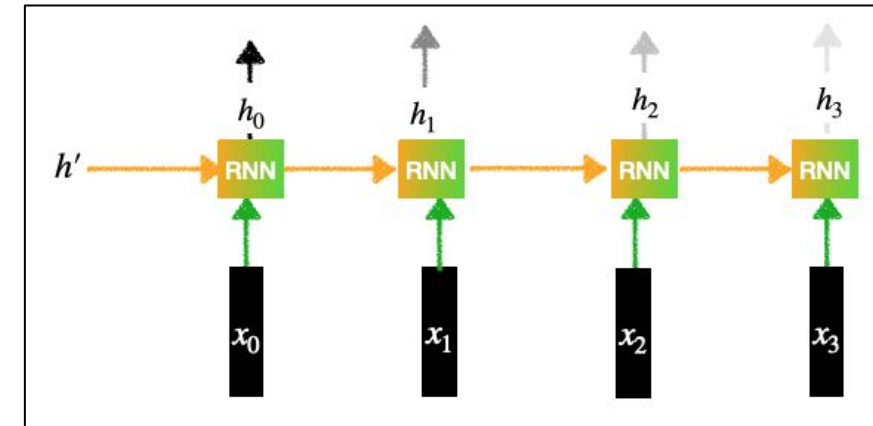
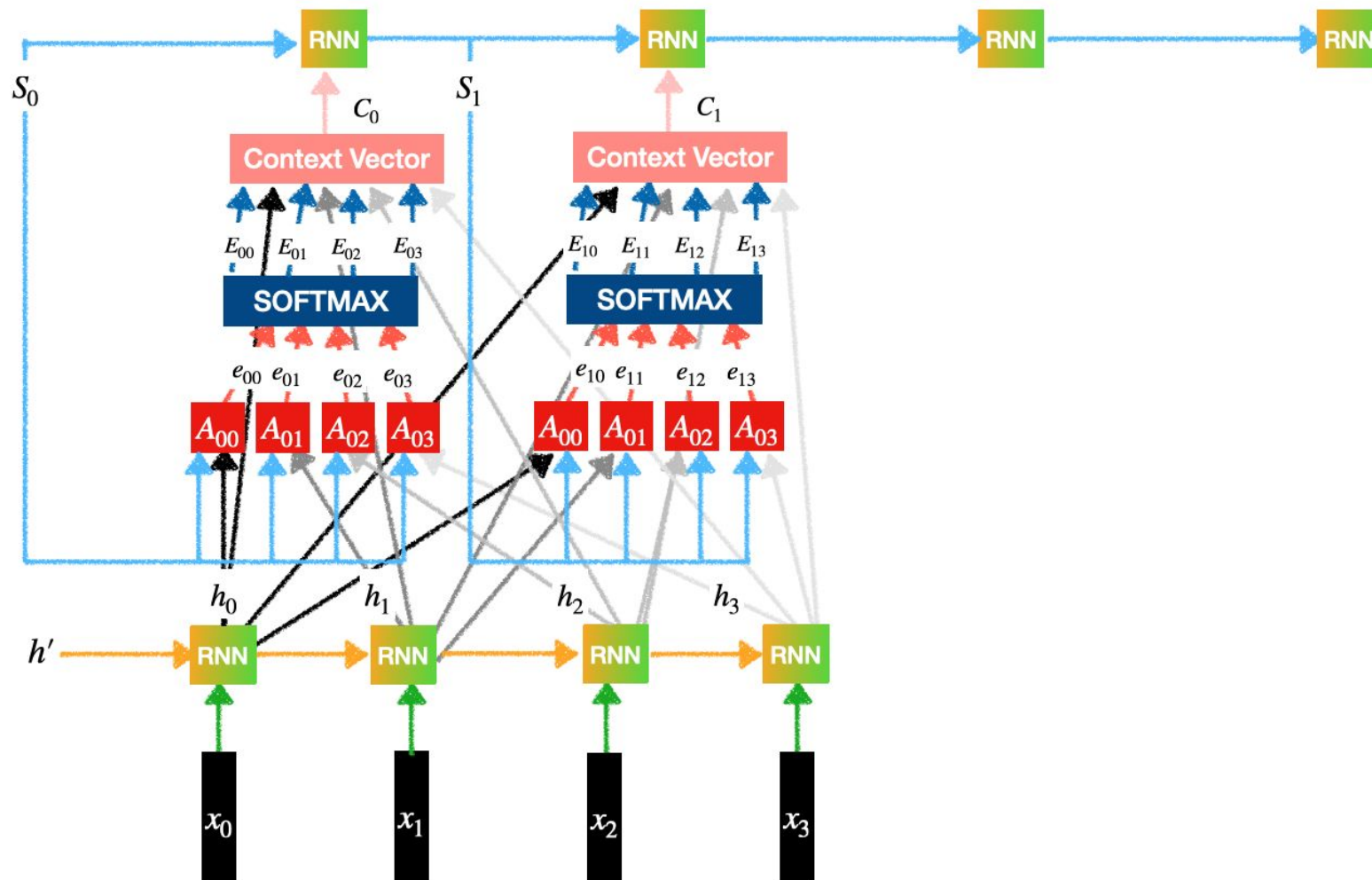
Source: "Findings of the 2018 Conference on Machine Translation (WMT18)", Bojar et al. 2018, <http://www.statmt.org/wmt18/pdf/WMT028.pdf>

Source: "Findings of the 2019 Conference on Machine Translation (WMT19)", Barrault et al. 2019, <http://www.statmt.org/wmt18/pdf/WMT028.pdf>

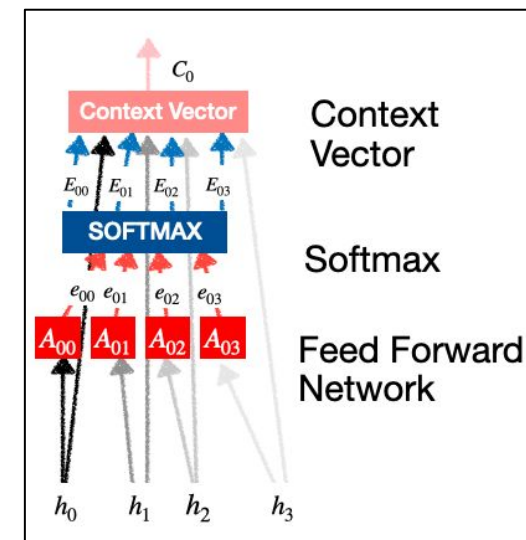
Attention Mechanism



Attention Mechanism

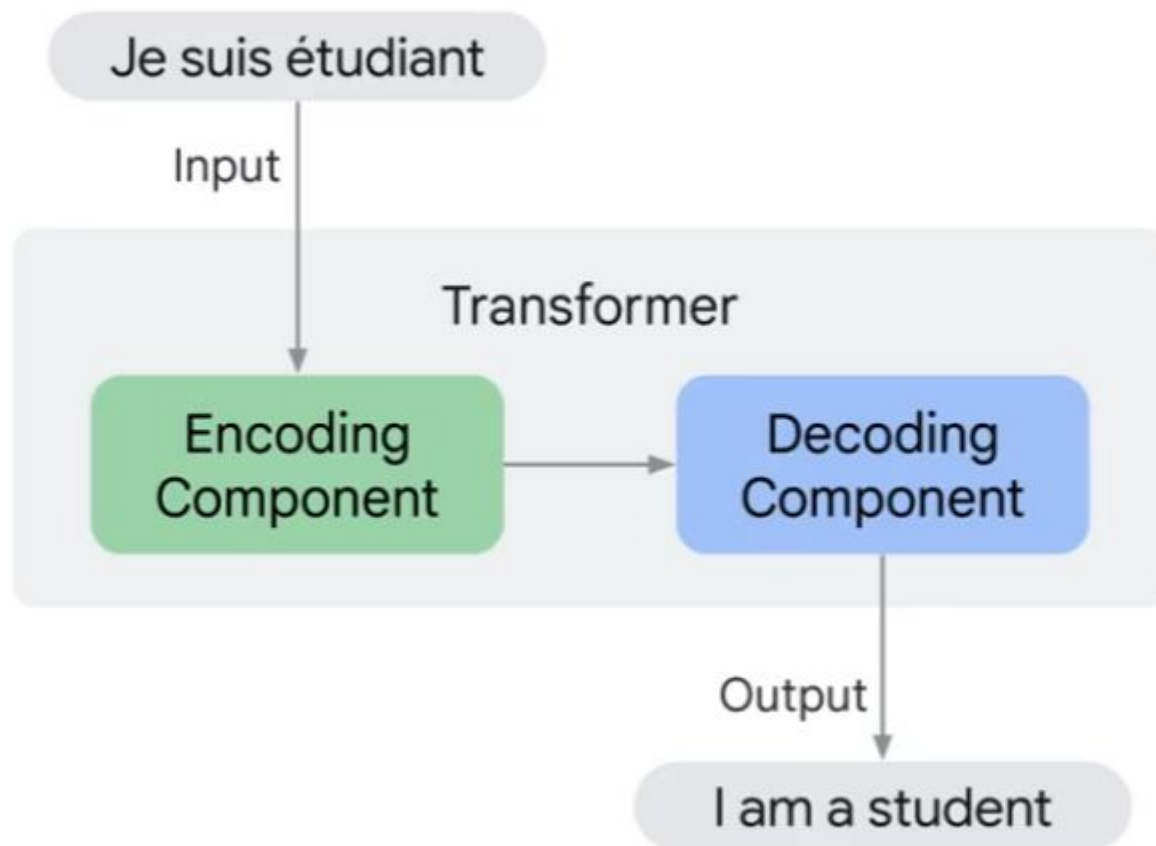


Encoder



Attention

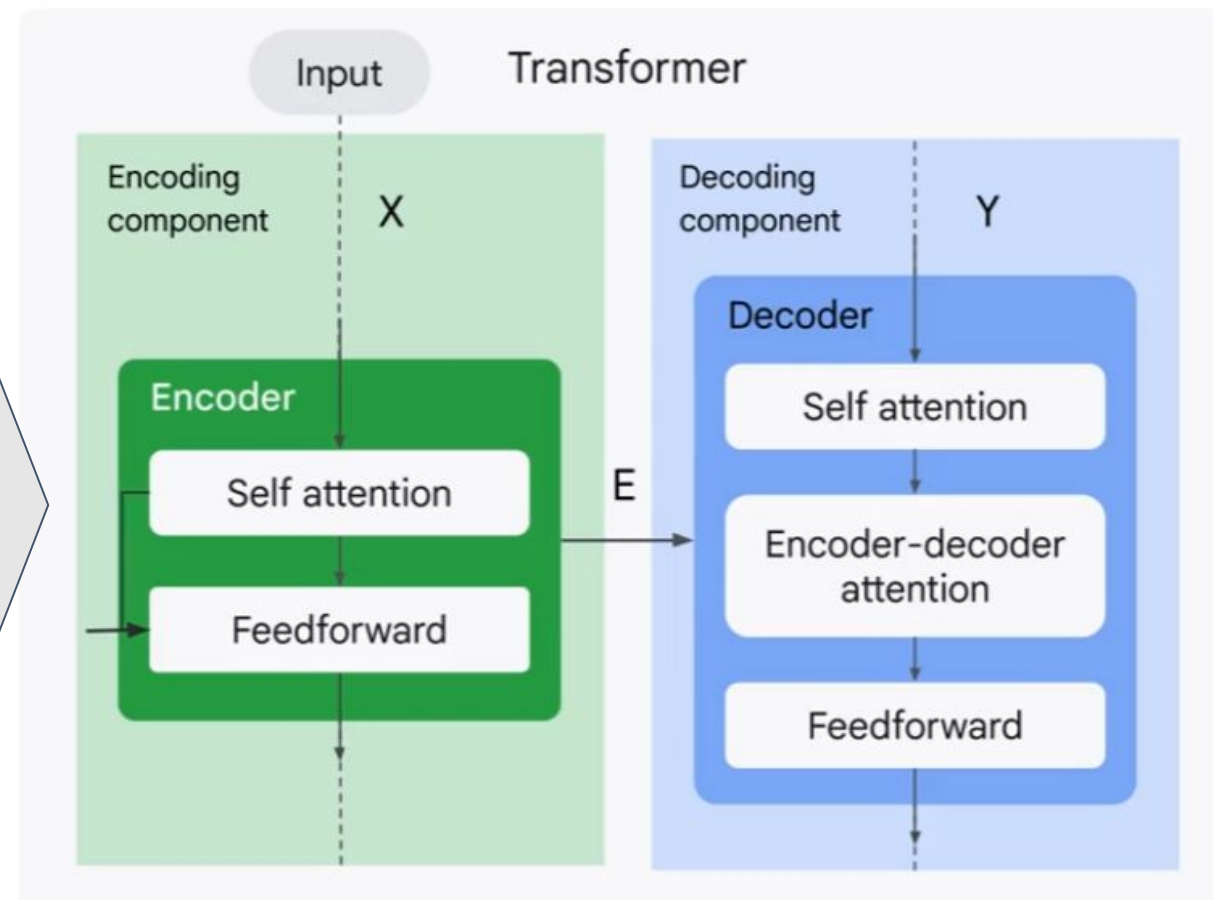
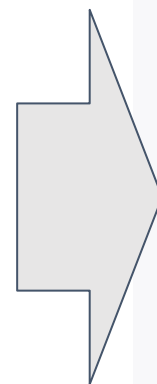
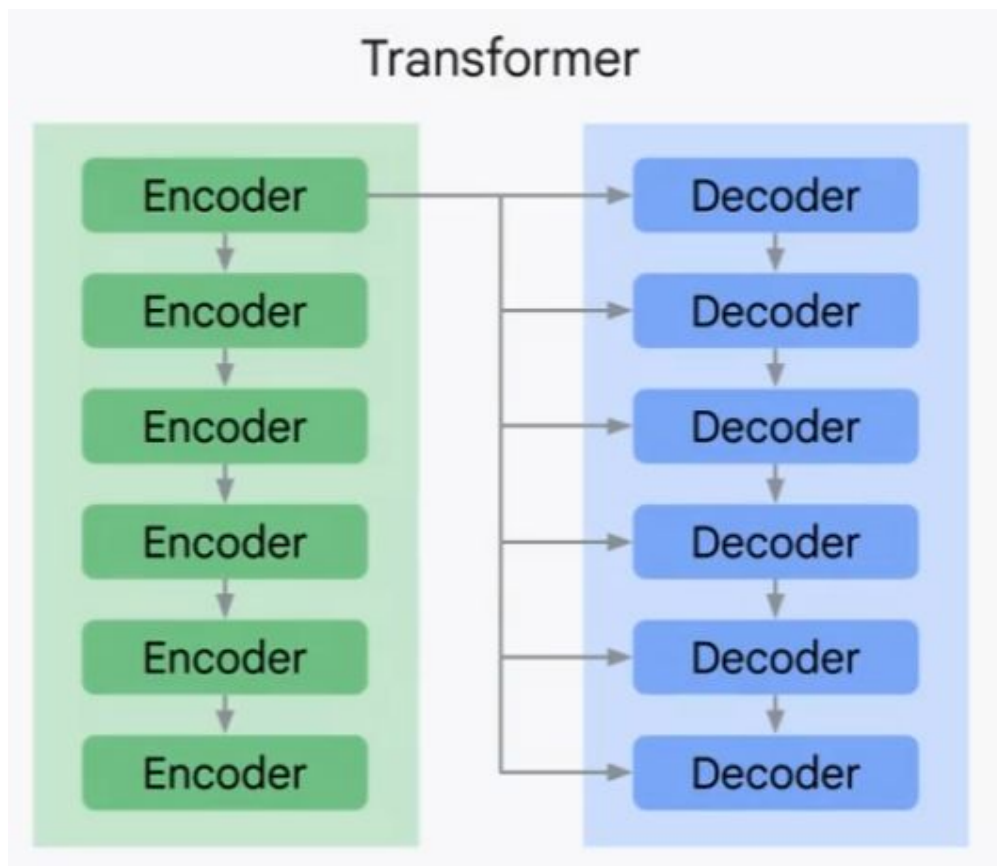
Transformer model



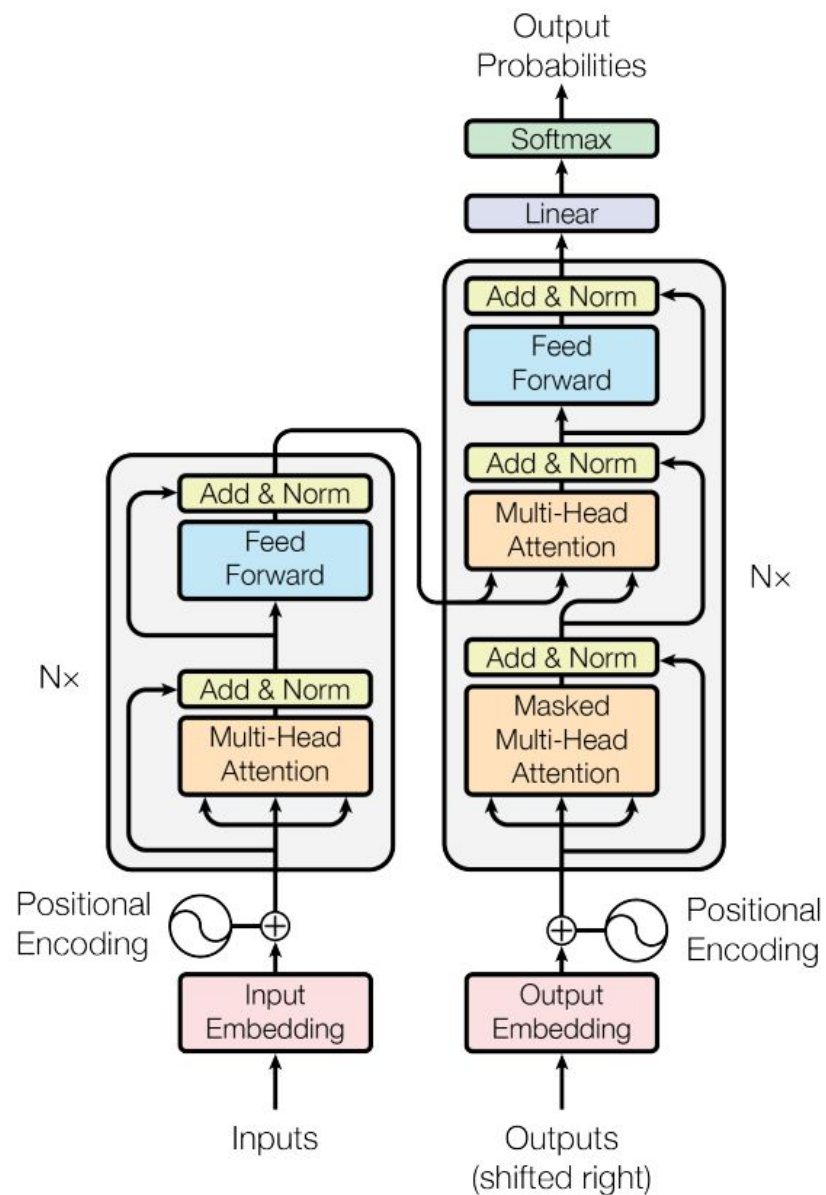
Massive advantage over RNN based encoder-decoder architecture since it allows us to:

- Take advantage of parallelization GPU/TPU.
- Process much more data in the same amount of time.
- Process all tokens at once!

Transformer model



Transformer model



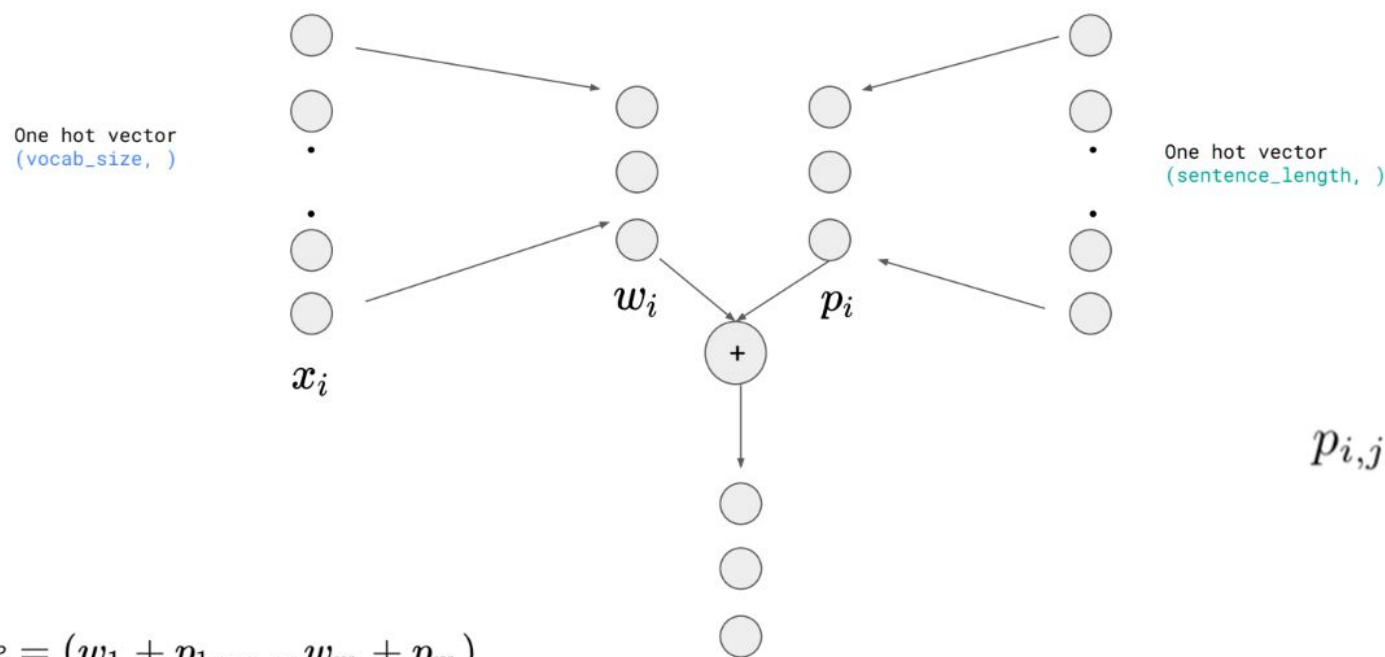
The Transformer - model architecture

<https://arxiv.org/pdf/1706.03762.pdf>

Input Embedding

Learned positional embeddings

Learned positional embeddings



$$e = (w_1 + p_1, \dots, w_m + p_m)$$

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

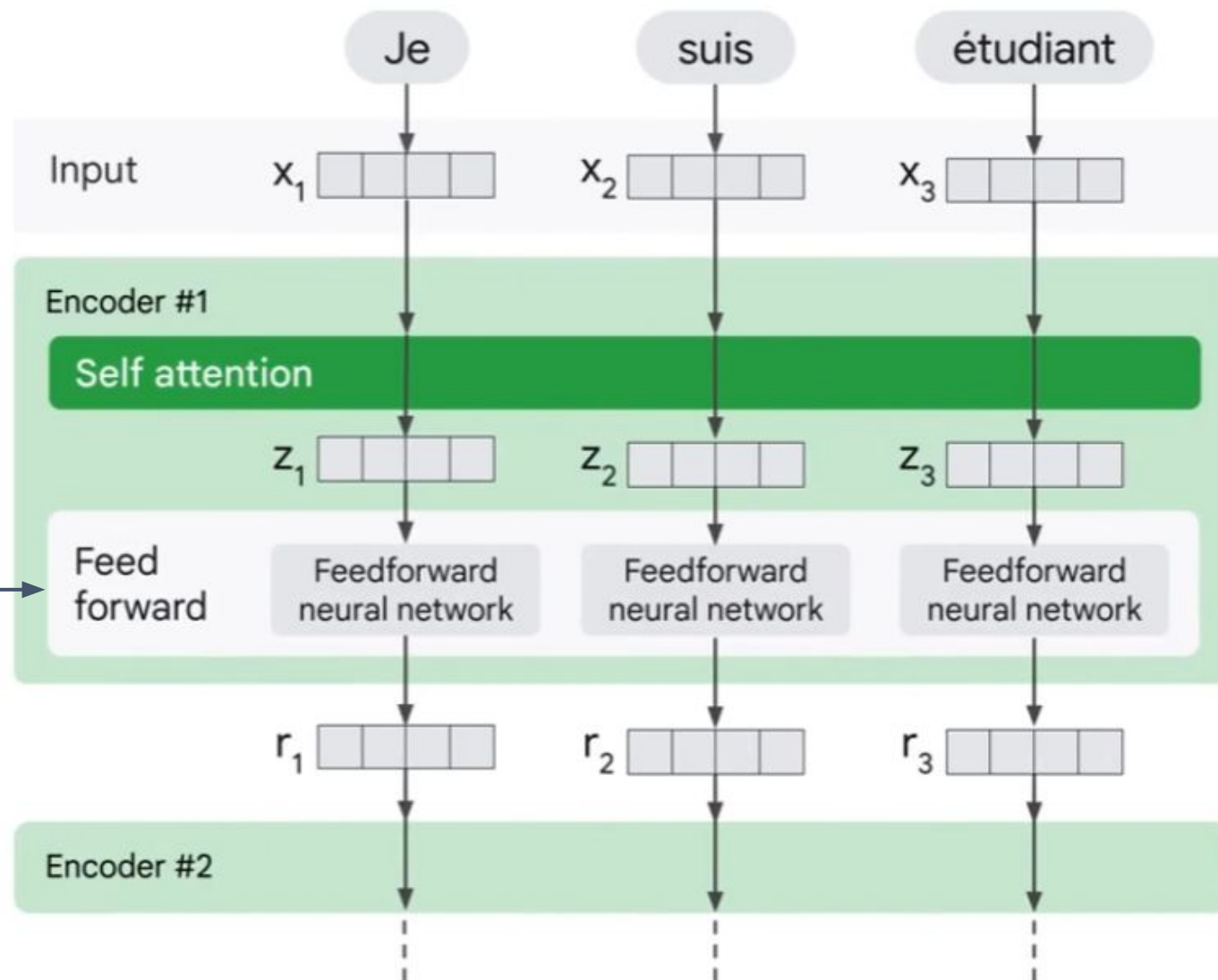
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

$$p_{i,j} = \begin{cases} \sin\left(\frac{i}{10000^{\frac{j}{d_{\text{emb-dim}}}}}\right) & \text{if } j \text{ is even} \\ \cos\left(\frac{i}{10000^{\frac{j-1}{d_{\text{emb-dim}}}}}\right) & \text{if } j \text{ is odd} \end{cases}$$

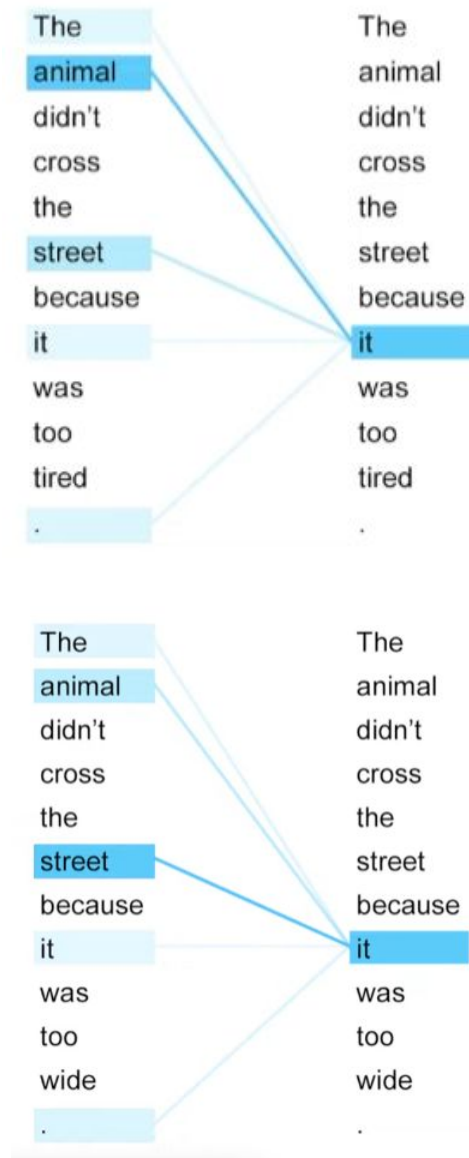
i position in sentence

j position in vector P (ex: dimension 512)

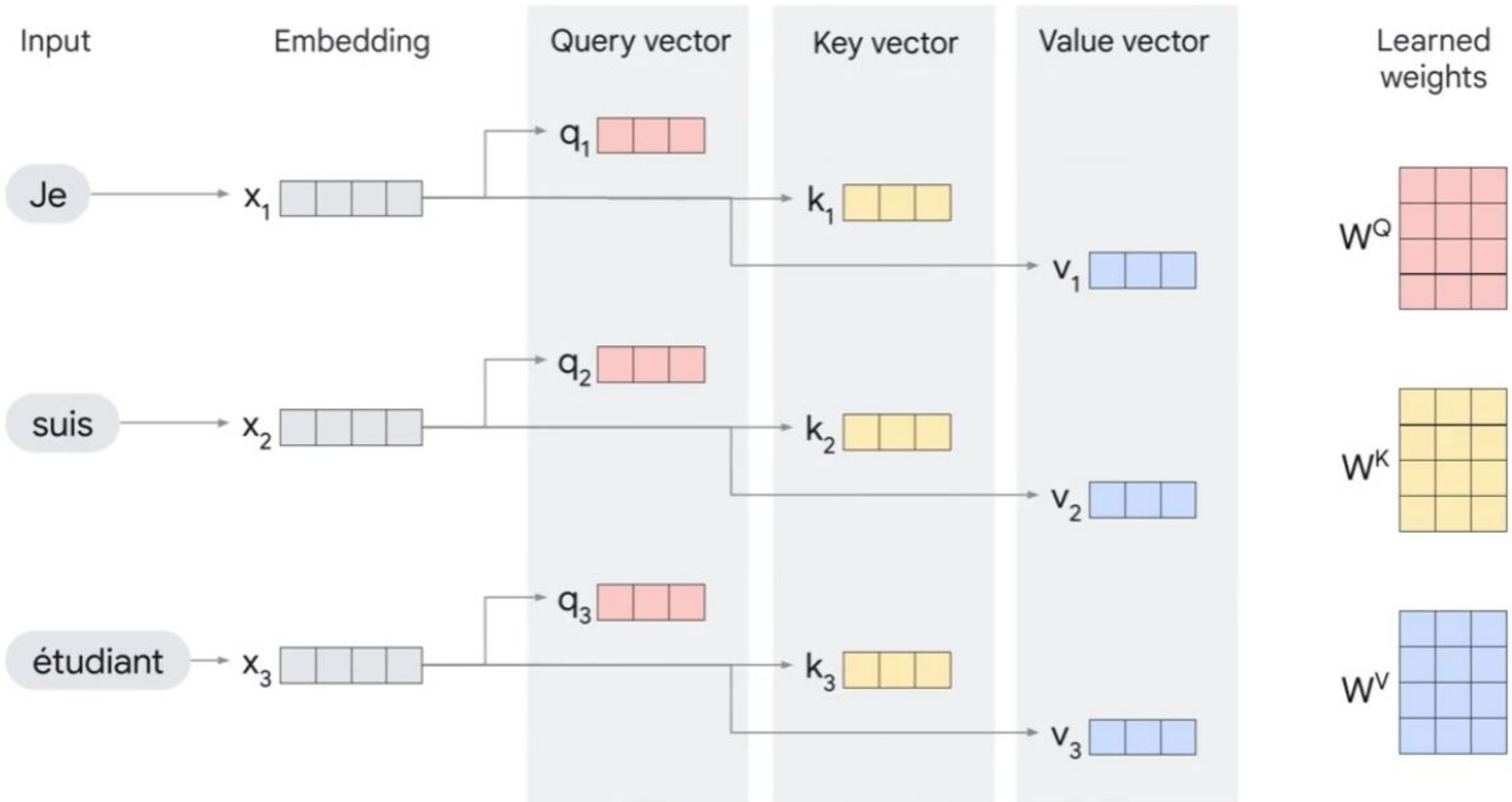
Encoder Part



Allows for
parallelization



Encoder Part



Encoder Part

Input

Je

suis

étudiant

Embedding

x

x_1				
x_2				
x_3				

Query vector

Q

q_1			
q_2			
q_3			

Key vector

K

k_1			
k_2			
k_3			

Value vector

V

v_1			
v_2			
v_3			

Learned weights

W^Q

W^K

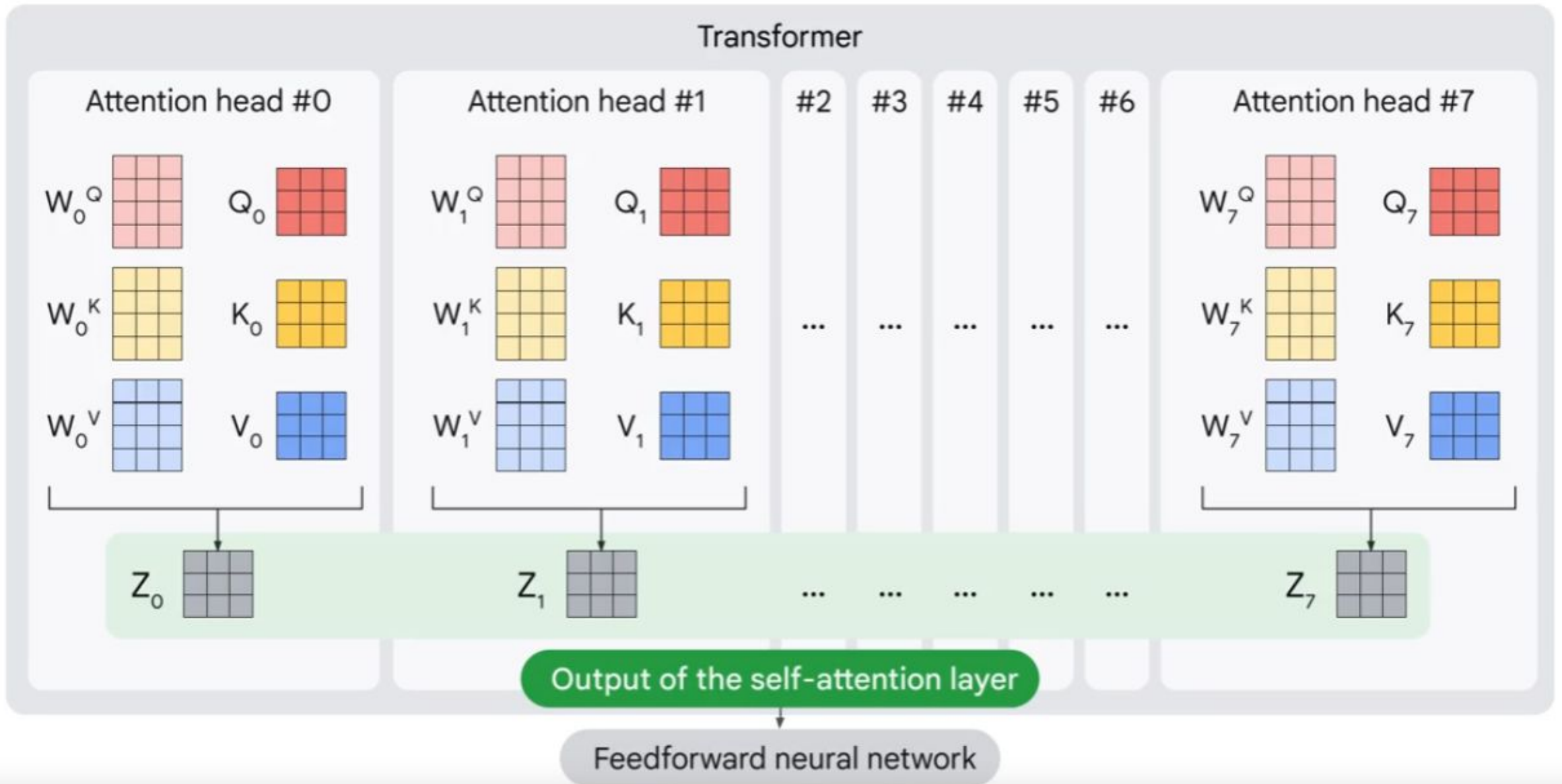
W^V

Encoder Part

$$\text{softmax}_x \left[\frac{\begin{matrix} Q & K^T \\ \begin{matrix} \text{3x3 red grid} & \times & \begin{matrix} \text{3x3 yellow grid} \end{matrix} \end{matrix}}{\sqrt{d_k}} \right] V = Z$$

The diagram illustrates the encoder part of a transformer block. It shows the calculation of the output Z as the product of the scaled dot-product attention matrix and the value matrix V . The attention matrix is derived from the query matrix Q (3x3 red grid) and the key matrix K^T (3x3 yellow grid) via a softmax operation over the keys, scaled by $\sqrt{d_k}$. The result is a 3x3 blue grid, which is then multiplied by the value matrix V (3x3 blue grid) to produce the final output Z (3x3 gray grid).

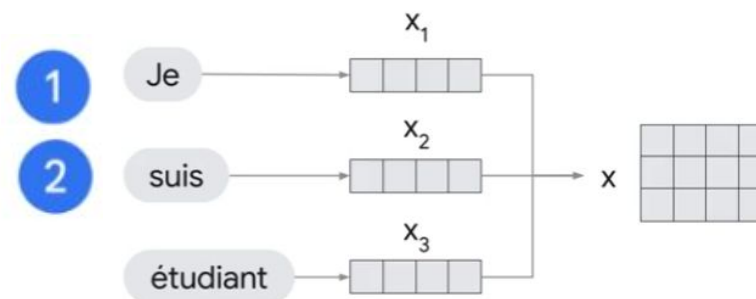
Encoder Part



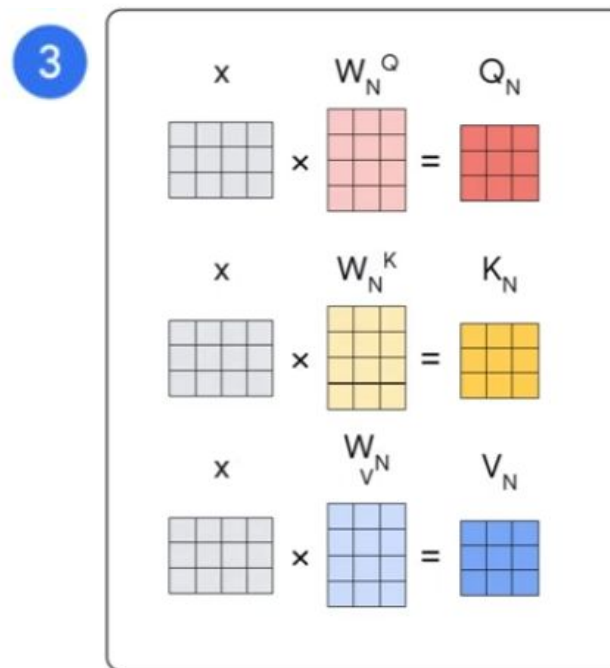
Encoder Part - Summarize

Process of getting the final **z** embeddings:

- 1 Input natural language sentences.
- 2 Embed each word.
- 3 Perform multi-headed attention, and multiple the embedded words with the respective weight matrices.
- 4 Calculate the attention using the resulting QKV matrices.
- 5 Concatenate the matrices to produce the output matrix which is the same dimension as the final matrix.



For each attention head:

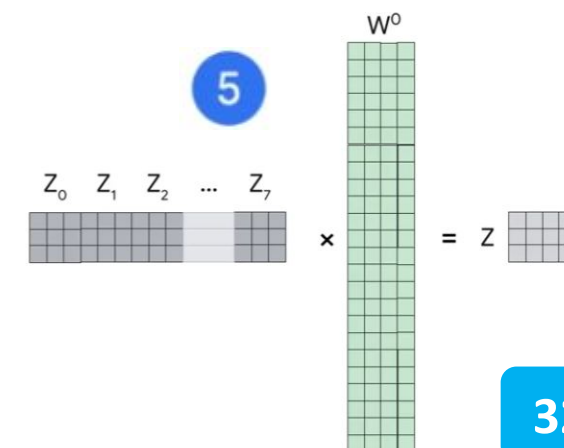


$N = 8$

For each attention head:

$$\text{softmax} \left(\frac{Q_N \times K_N^T}{\sqrt{d_k}} \right) V_N = Z_N$$

$N = 8$

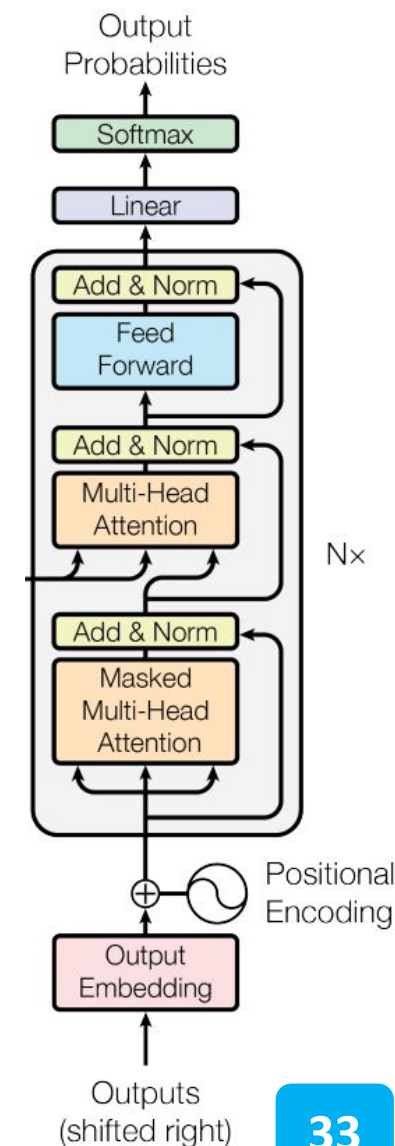


Decoder Part

Masked multi-head attention means the multi-head attention receives inputs with masks so that the attention mechanism does not use information from the hidden (masked) positions. The paper mentions that they used the mask inside the attention calculation by setting attention scores to negative infinity (or a very large negative number). The softmax within the attention mechanisms effectively assigns zero probability to masked positions.

Intuitively, it is as if we were gradually increasing the visibility of input sentences by the masks:

```
(1, 0, 0, 0, 0, ..., 0) => (<SOS>)  
(1, 1, 0, 0, 0, ..., 0) => (<SOS>, 'Bonjour')  
(1, 1, 1, 0, 0, ..., 0) => (<SOS>, 'Bonjour', 'le')  
(1, 1, 1, 1, 0, ..., 0) => (<SOS>, 'Bonjour', 'le', 'monde')  
(1, 1, 1, 1, 1, ..., 0) => (<SOS>, 'Bonjour', 'le', 'monde', '!')
```



BERT

Pre-trained transformer models

Encoder & Decoder



Decoder only

GPT-3

GPT-2

Encoder only



2018 - Google

Bidirectional

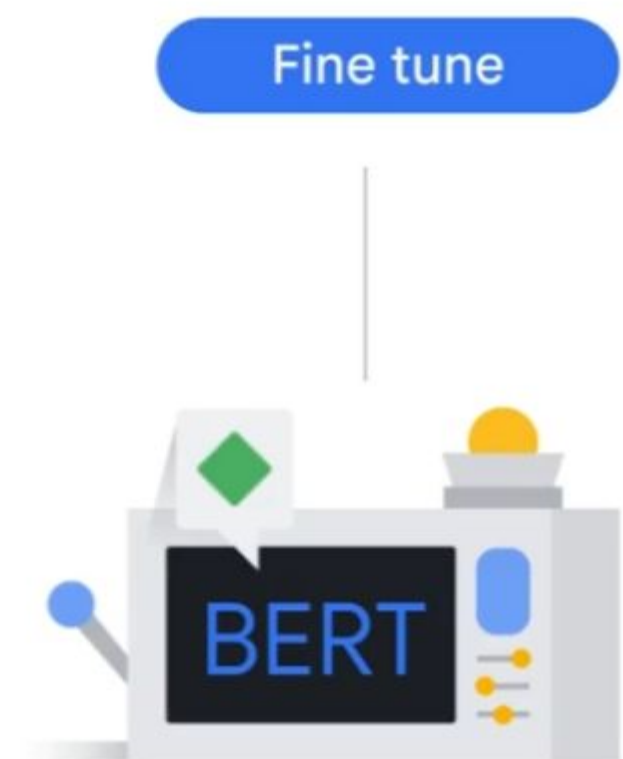
Encoder

Representation from

Transformer

BERT Overview

- Trained in two variations
- Able to handle long input context
- Trained on entire Wikipedia and BookCorpus
- Trained for one million steps
- Targeted at multi-task objective
- Trained on TPU
- Works at both sentence-level and token-level tasks
- Be fine-tuned for many different tasks



BERT version

	BERT _{BASE}	BERT _{LARGE}	Transformer
Layers	12	24	6
Feedforward networks (hidden units)	768	1024	512
Attention heads	12	16	8

110m parameters 340m parameters

BERT tiny, L=2, H=128
BERT mini, L=4, H=256
BERT small, L=4, H=512
BERT medium, L=8, H=512

LLM AI Model	Parameters	Year
BERT	340 million	2018
GPT-2	1.5 billion	2019
Meena	2.6 billion	2020
GPT-3	175 billion	2020
LaMDA	137 billion	2022
BLOOM	176 billion	2022

LLM models parameter

BERT - Mask Language Modeling (MLM)

Mask out $k\%$ of the input words, and then predict the masked words

- Recommendation use $k = 15\%$

The man went to the [MASK] to buy a [MASK] of milk.

store

gallon

Too little masking

Too expensive to train

Too much masking

Not enough context

BERT - Next Sentence Prediction (NPS)

Learn the relationships between sentences and predict the next sentence given the first one.

Binary classification task

Sentence A	The man went to the store.
------------	----------------------------

Sentence B	He bought a gallon of milk.
------------	-----------------------------

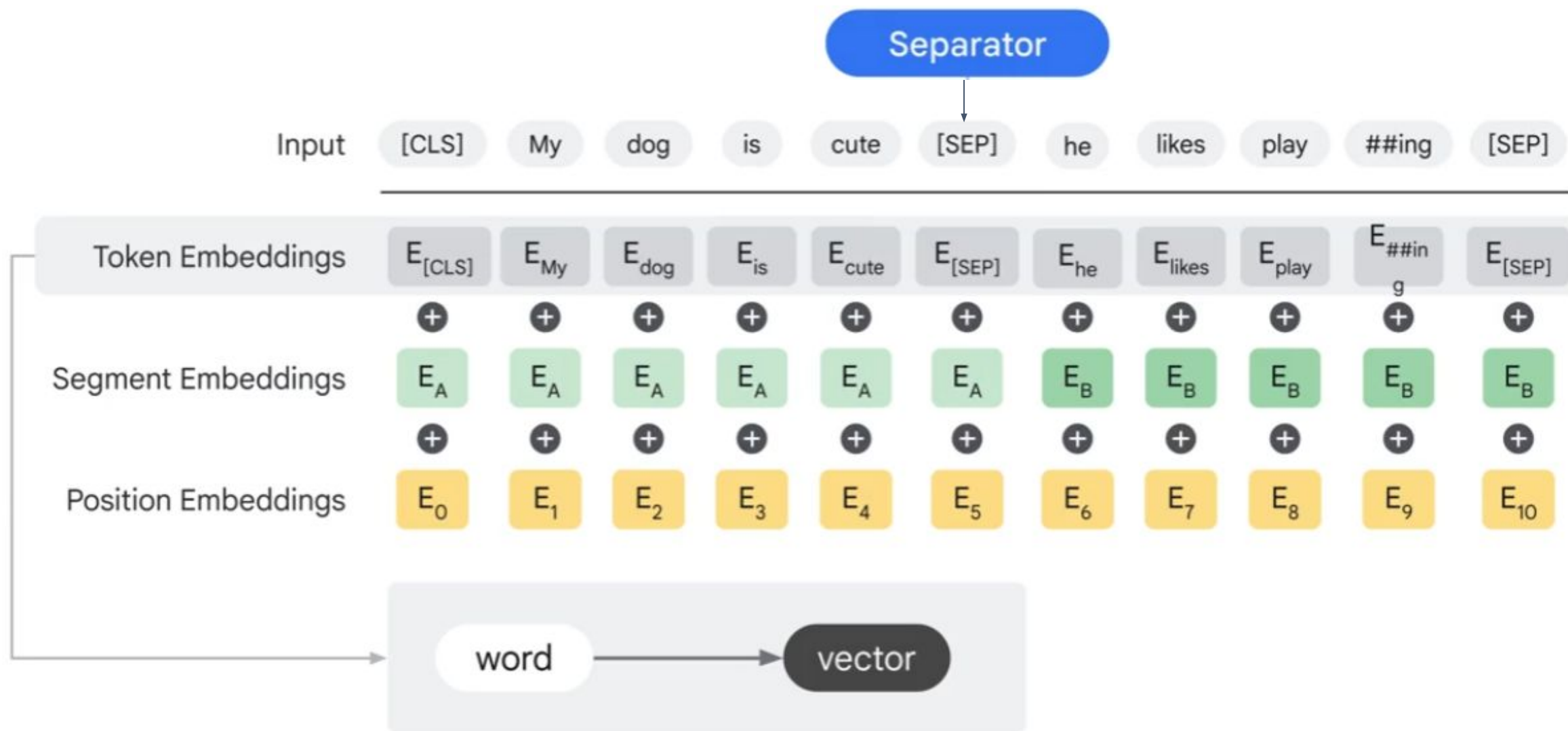
Label	IsNextSentence
-------	----------------

Sentence A	The man went to the store.
------------	----------------------------

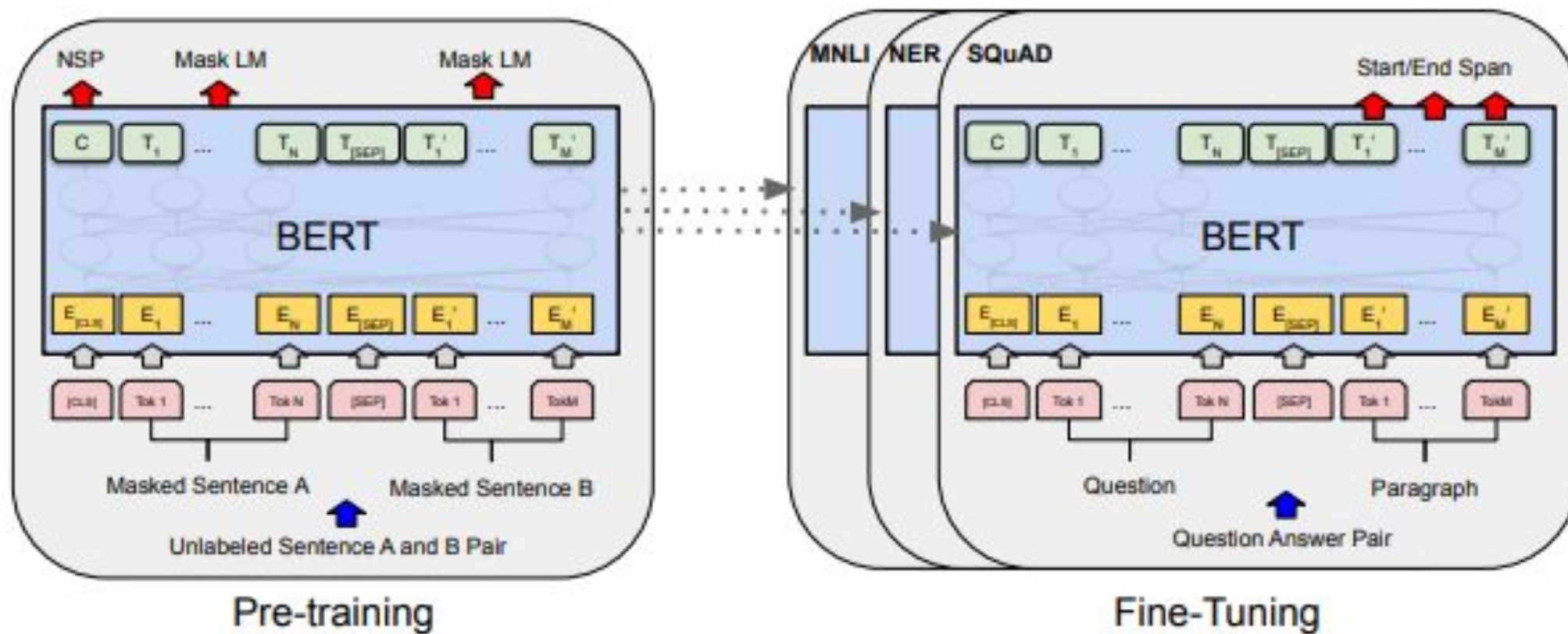
Sentence B	Penguins are flightless.
------------	--------------------------

Label	NotNextSentence
-------	-----------------

BERT - Input Embedding



BERT - Pre-training

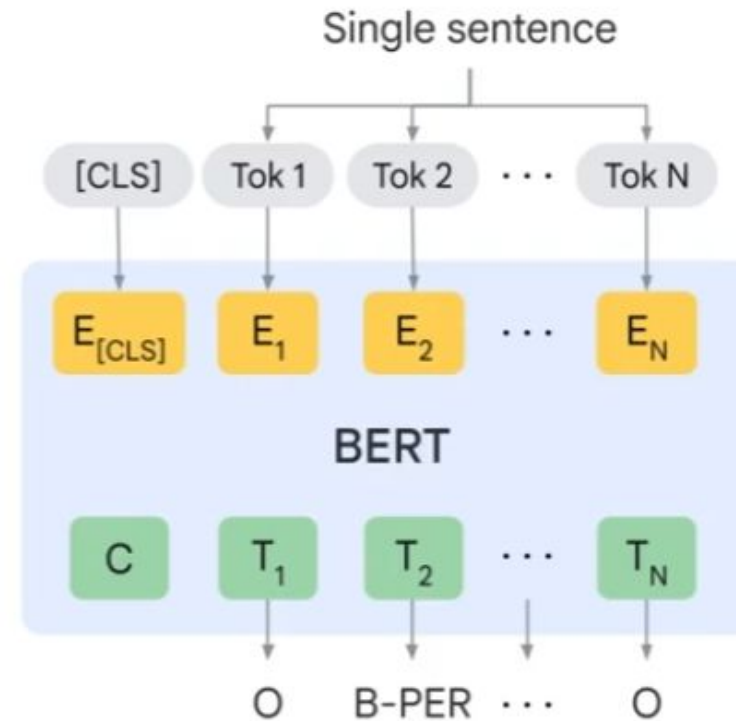


BERT - Downstream Tasks

You can use BERT for various downstream tasks, or example:

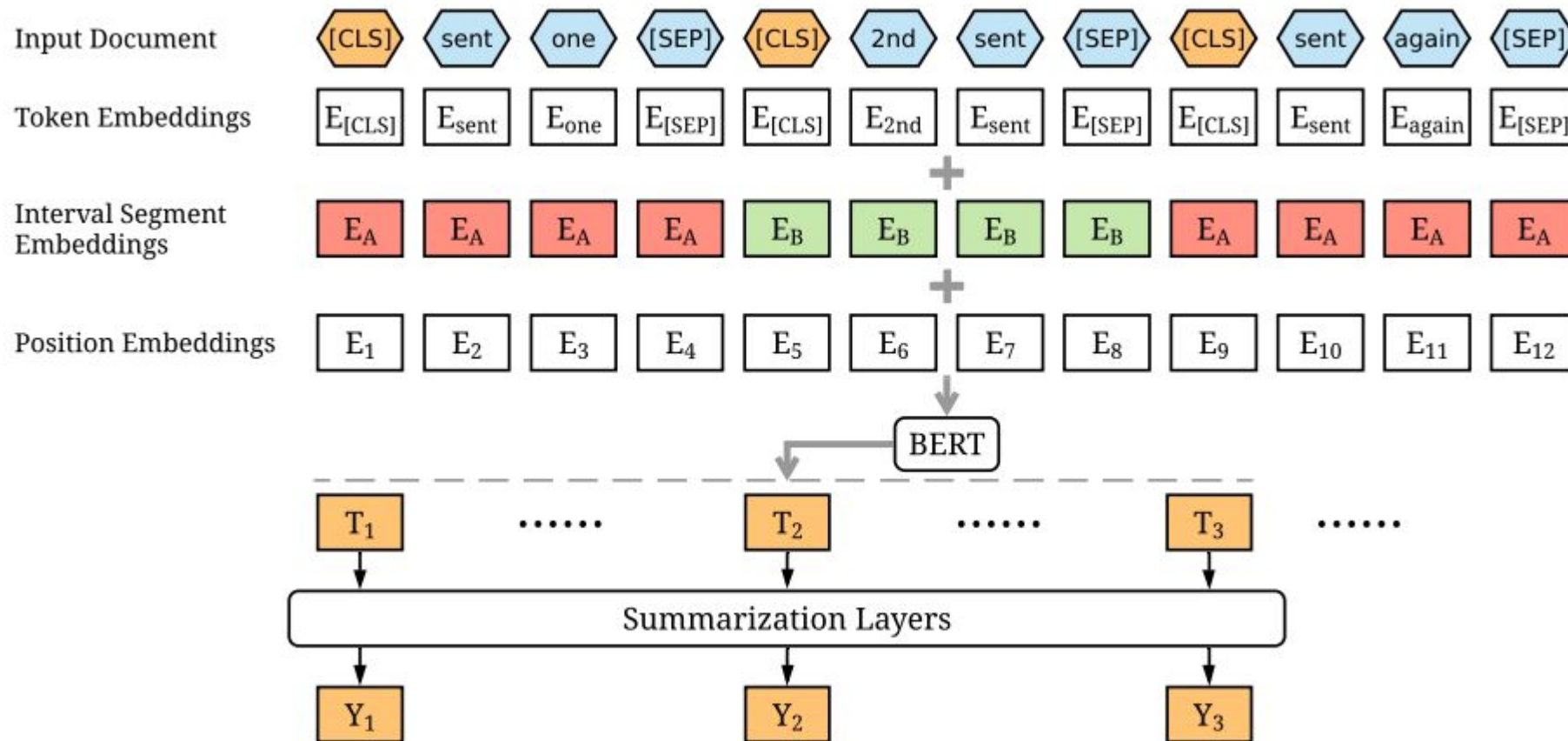
- ✓ Single sentence classification
- ✓ Sentence pair classification
- ✓ Question answering
- ✓ Single sentence tagging tasks

Single sentence tagging tasks: CoNLL-2003 NER



HAND ON - BERT

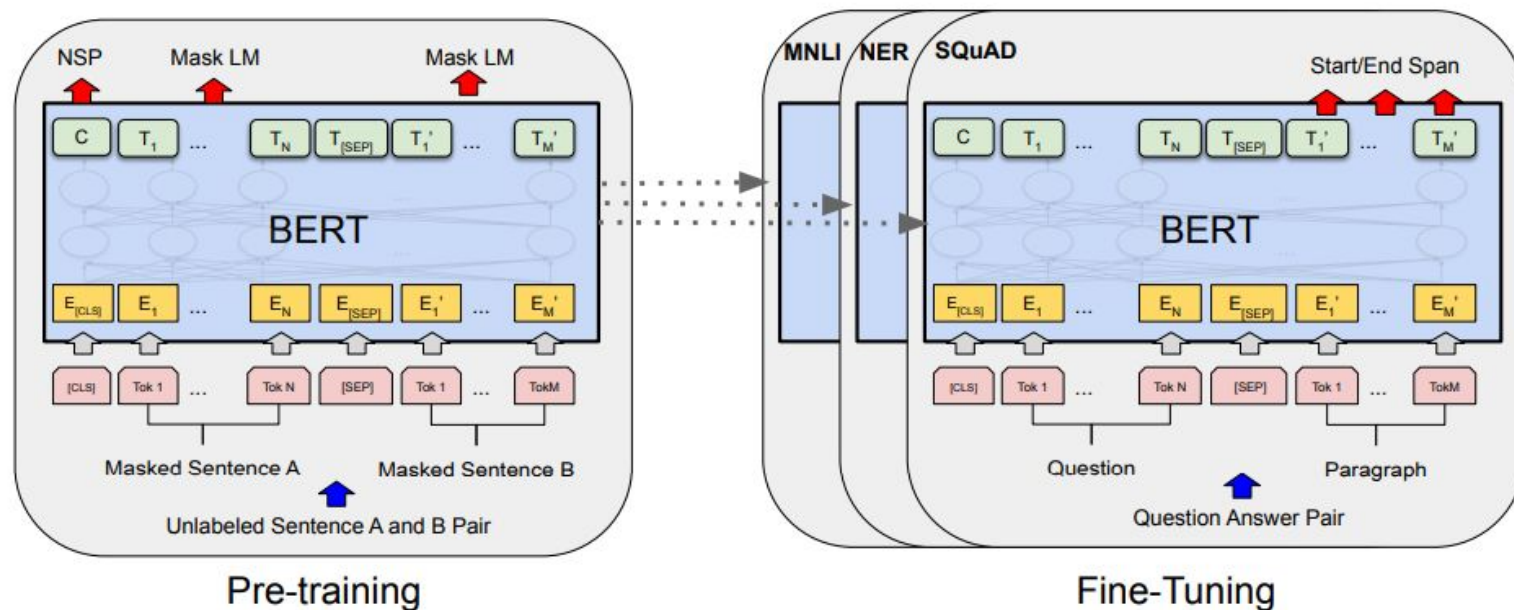
Hand On - Text Summarization - BERTSUM



<https://arxiv.org/pdf/1903.10318.pdf>

<https://github.com/huggingface/notebooks/blob/main/examples/summarization-tf.ipynb>

Hand On - Bert Question Answering



BERT SQuAD Architecture

To perform the QA task we add a new question-answering head on top of BERT, just the way we added a masked language model head for performing the MLM task. The purpose of this question-answering head is to find the start token and end token of an answer for a given paragraph, for example:

“ Paragraph: BERT-large is really big ... it has 24 layers and an embedding size of 1024 for a total of 340M parameters! Altogether it is 1.34GB, so expect it to take a couple of minutes to download.
Question: How many parameters does BERT-large have?
Answer: 340M parameters
Start token: 340
End token: parameters

<https://blog.paperspace.com/how-to-train-question-answering-machine-learning-models/>
https://github.com/huggingface/notebooks/blob/main/examples/question_answering_tf.ipynb
https://github.com/Dorink/BERT-for-Question-Answering-task/blob/main/BERT_For_Question_Answering_Model_2.ipynb

THANK YOU!



Điện thoại: (028) 3997 8000 – Ext: 5945



Email: contact@tmainnovation.vn



Website: www.tmainnovation.vn

